252-0027

Einführung in die Programmierung
Übungen

Woche 7: Hoare-Logik, Problem Solving

Fabian Rohlik
Departement Informatik
ETH Zürich

Heute

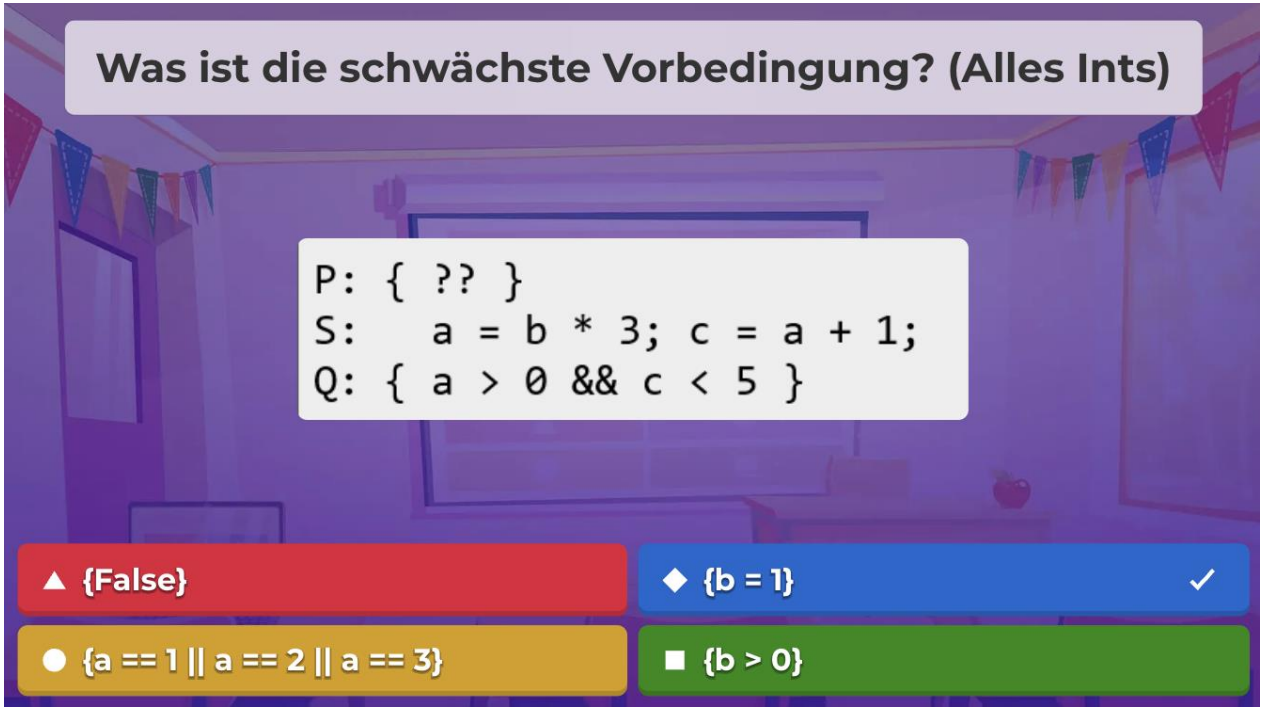
- Weakest Precondition mit Verzweigung
- Hoare Triple
- Rekursion III
- Timed Bonus & Vorbesprechung

Timed Bonus

- Release: 03.11.2026 17:00 – 19:00
- Simuliert die Prüfung unter Zeitdruck
- statt der (normalen) wöchentlichen Bonusaufgabe

Weakest Precondition

```
P: { ?? }
S:    a = b * 3; c = a + 1;
Q: { a > 0 && c < 5 }
```


Was ist die schwächste Vorbedingung? (Alles Ints)
P: { ?? }
S: a = b * 3; c = a + 1;
Q: { a > 0 && c < 5 }
▲ {False}
◆ {b = 1} ✓
● {a == 1 || a == 2 || a == 3}
■ {b > 0}

Weakest Precondition mit Verzweigung

Beispiele

Weakest Precondition – Mit Verzweigung

```
{              }
if (x<5) {
        y = x*x;
} else {
        y = x+1;
}
```

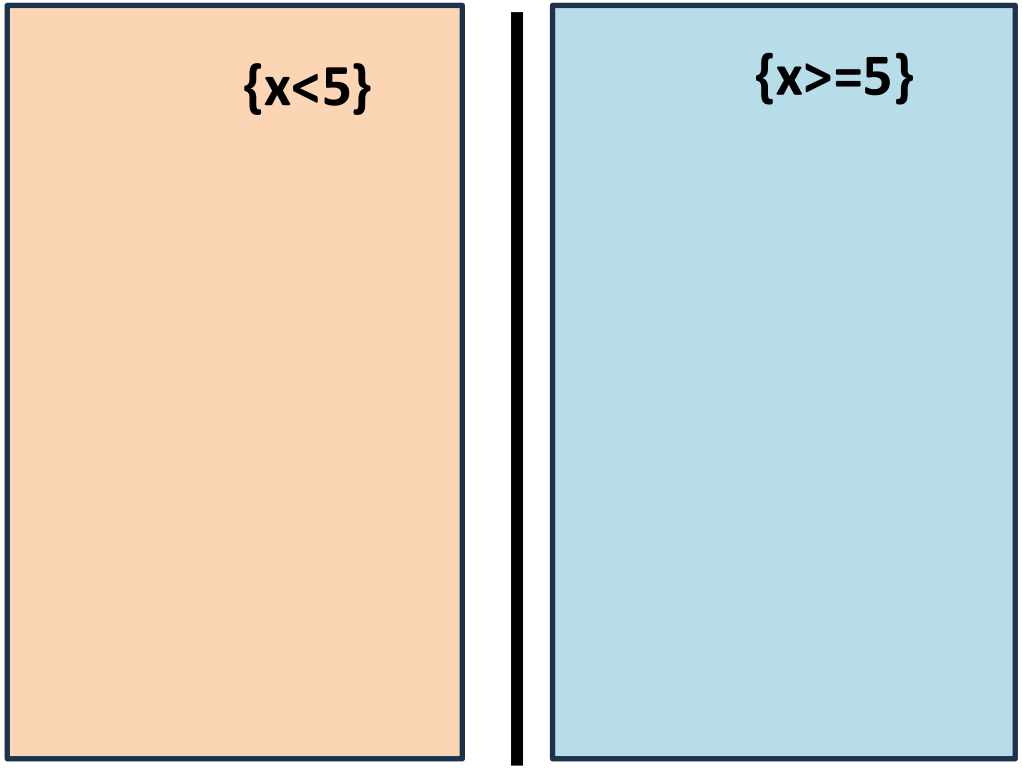
{y>9}

Weakest Precondition – Mit Verzweigung

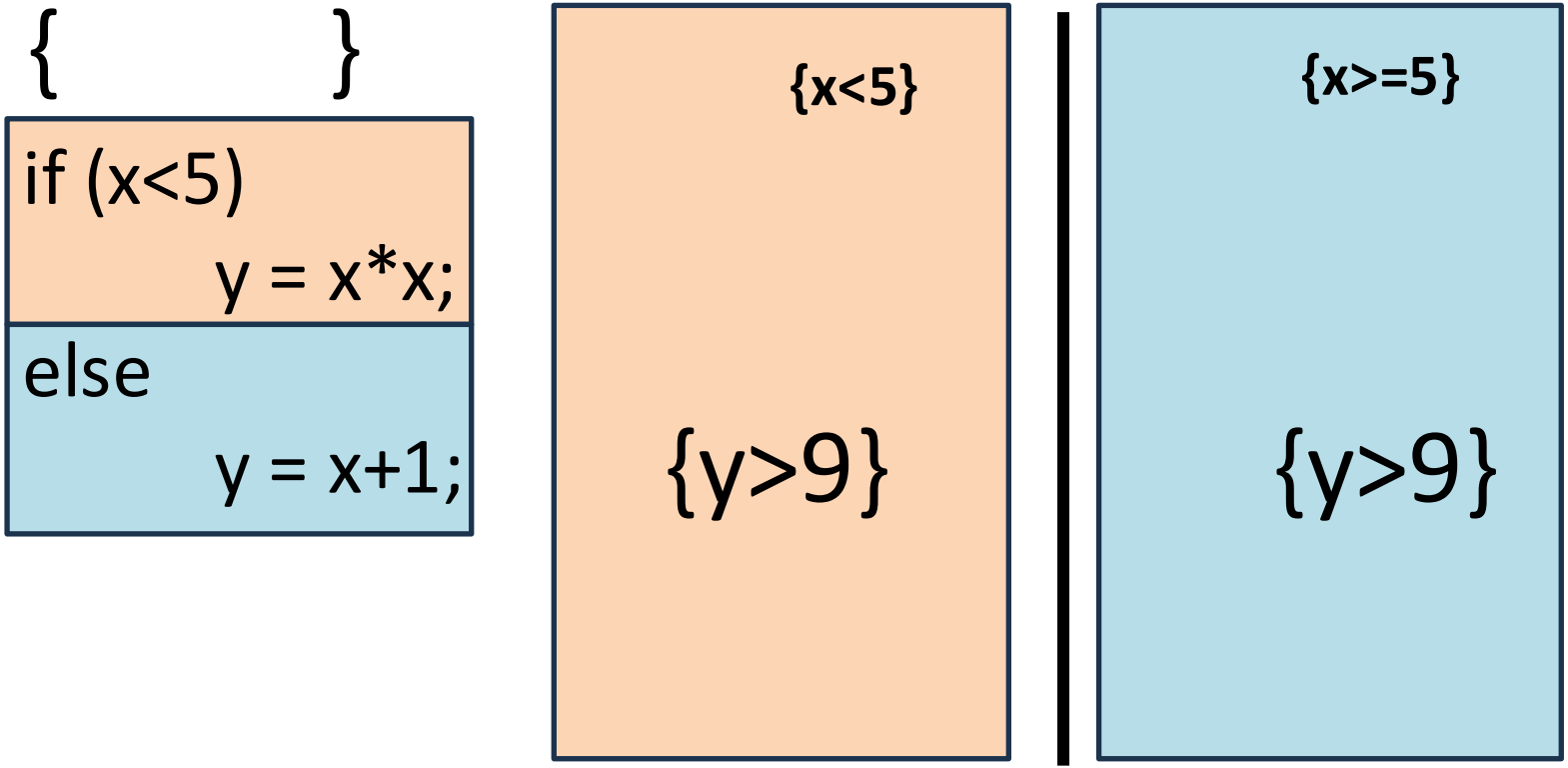
{ }

```
if (x<5)
        y = x*x;
else
        y = x+1;
```

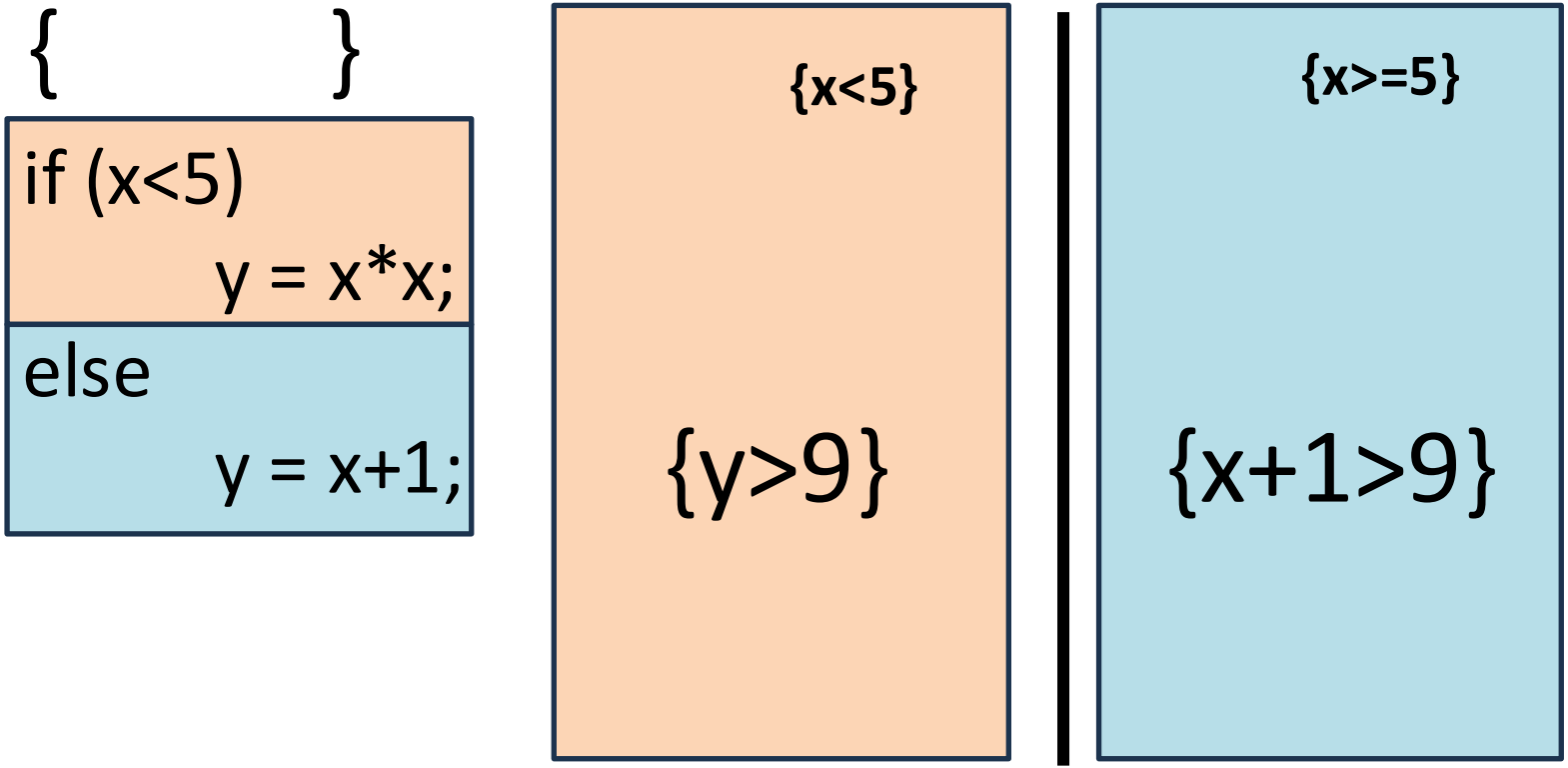
{y>9}



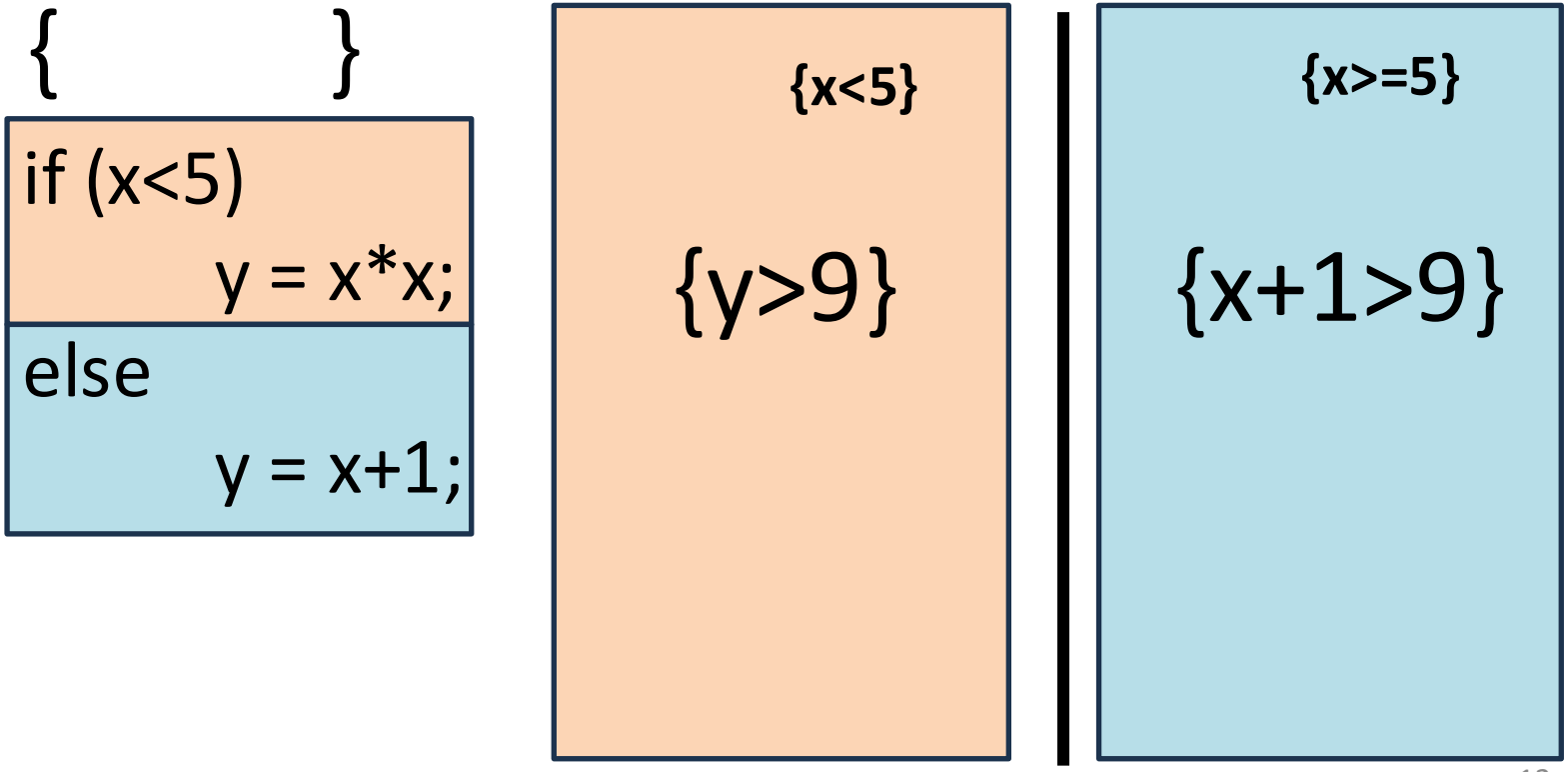
Weakest Precondition – Mit Verzweigung



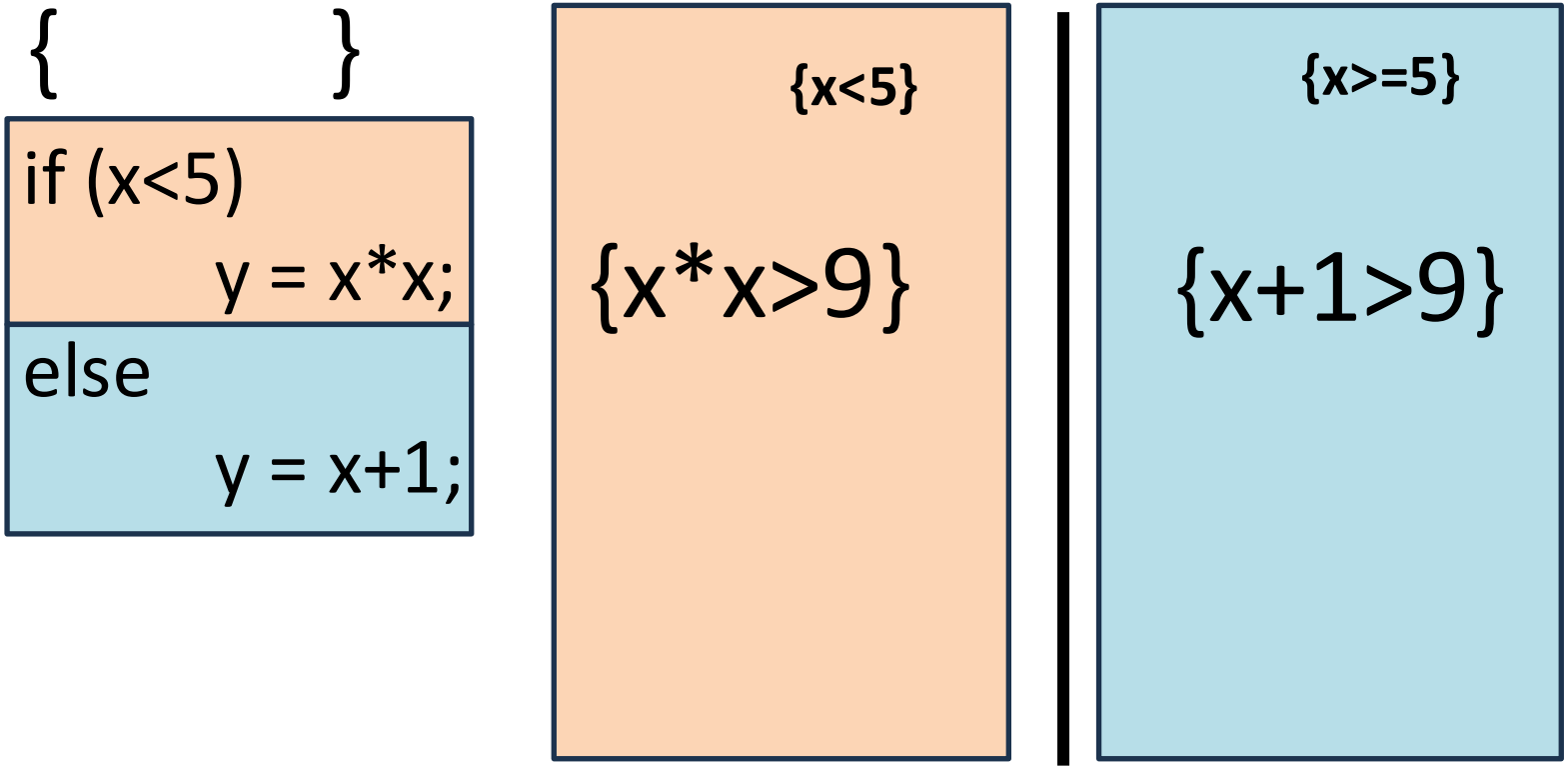
Weakest Precondition – Mit Verzweigung



Weakest Precondition – Mit Verzweigung



Weakest Precondition – Mit Verzweigung



{(**x<5** && x*x>9) || (**x>=5** && x+1>9)}

{ }

```
if (x<5)
        y = x*x;
else
        y = x+1;
```

{x<5}

{x*x>9}

{x>=5}

{x+1>9}

Weakest Precondition - Prüfungsbeispiel

```
{              }
 if (x > y)
        z = x – y;
 else
        z = y – x;

{  z > 0  }
```

{((**x > y**) && (x > y)) || (**(x <= y)** &&(y > x))}

{ }

```
if (x > y)
        z = x – y;
else
        z = y – x;
```

{ z > 0 }

{x > y}

{x > y}

{x <= y}

{y > x}

Schwächste Vorbedingung - Beispiel

```
{              }

  if (x >= y){
    y = x
  }

{y >= x}
```

Schwächste Vorbedingung - Beispiel

```
{              }

  if (x >= y){
    y = x
  }

{y >= x}
```

ohne Formalismus, was könnte die schwächste Vorbedingung sein?

Schwächste Vorbedingung - Beispiel

```
{  true  }

  if (x >= y){
    y = x
  }

{y >= x}
```

ohne Formalismus, was könnte die schwächste Vorbedingung sein?

Weakest Precondition mit Verzweigung

Theorie

If-Anweisungen und Aussagen

- **Beobachtungen**
  - Ausführung von S_1 wenn b hält
  - Ausführung von S_2 wenn ¬b hält
  - P hält in beiden Fällen vor der Ausführung
  - Q muss in beiden Fällen nachher gelten

- Das Tripel `{P} if (b) S1 else S2 {Q}` ist gültig, genau dann wenn
  1. $\{P \wedge b\}\ S_1\ \{Q\}$ gültig ist
  2. $\{P \wedge \neg b\}\ S_2\ \{Q\}$ gültig ist

```
{P}
  if (b) {
    S1;
  } else {
    S2;
  }
{Q}
```

Weakest Precondition

- Falls die Postcondition **`{ true }`** ist, so ist **`{ true }`** die schwächste Vorbedingung. Alles impliziert die Postcondition, also insbesondere auch die schwächste Bedingung `true`.
- Falls die Postcondition **`{ false }`** ist, so ist **`{ false }`** die schwächste Vorbedingung. Nur **`{ false }`** impliziert die Postcondition, demensprechend ist es die schwächste (und einzige) Vorbedingung.

- Die vorgestellten Regeln fürs Rückwärtsschliessen ergeben direkt die schwächsten Vorbedingungen.

True ist die schwächere Aussage als False. False > True, da False => alles. PC true = alles ok. PC false = logisch unmöglich.

Hoare Logik Allgemein

Gültigkeit eines Triples prüfen

Wir betrachten die weakest precondition, und schauen, ob die gegebene precondition, die WP impliziert. $P \implies WP$

Die gegebene Vorbedingung P muss stärker (oder gleich stark) sein als die WP.

z.B. wenn wir als WP finden $x > 0$ und die gegebene precondition ist $x > 5$ (also stärker), ist die precondition korrekt, da

$x>5 \implies x>0$

Hoare Tripel

(ints)

```
{ x > 0 }
    y = x - 2;
    z = x * y;
{ z > 0 }
```

Hoare Tripel

(ints)

```
{ x > 0 }
    y = x - 2;
    z = x * y;
{ z > 0 }
```

ungültig

Hoare Tripel – Prüfungsbeispiel HS18

(ints)

```
{ x > 0 }
    y = x * x;
    z = y / 2;
{ z > 0 }
```

Hoare Tripel – Prüfungsbeispiel HS18

(ints)

```
{ x > 0 }
    y = x * x;
    z = y / 2;
{ z > 0 }
```

ungültig

Bei Ganzzahldivision aufpassen!!

Gültigkeit prüfen bei if/else

- **Gleiches Prinzip**, wir prüfen, ob die gegebene precondition die weakest precondition impliziert.
- Um das aber einfacher/übersichtlicher zu machen, betrachten wir die Implikationen der beiden if/else Teile getrennt

Gültigkeit prüfen bei if/else

- **Situation:** Entscheide, ob $\{P\}$ `if (b)` S_1 `else` S_2 $\{Q\}$ gültig ist

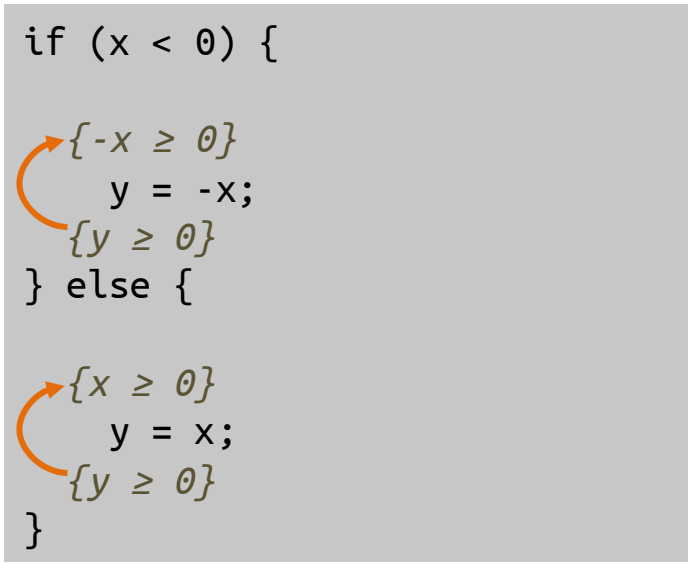
1. Wende bekannte Regeln des Rückwärtsschliessens an (weakest precondition)
2. Zeige notwendige Implikationen
   1. $P \wedge b \implies R_1$
   2. $P \wedge \neg b \implies R_2$
3. Nur, wenn **beide** Implikationen gelten, ist die precondition gültig

```
if (b) {
  {P ∧ b}
  {R1}
    S1;
  {Q}
} else {
  {P ∧ ¬b}
  {R2}
    S2;
  {Q}
}
```

Beispiel 1: Gültiges Tripel

Zu zeigen: Gültigkeit von $\{true\}$ `if (x < 0) { y = -x; } else { y = x;}` $\{y \geq 0\}$

1. Regeln anwenden:



Beispiel 1: Gültiges Tripel

Zu zeigen: Gültigkeit von $\{true\}$ `if (x < 0) { y = -x; } else { y = x;}` $\{y \geq 0\}$

1. Regeln anwenden:

```
if (x < 0) {
  {true ∧ x < 0}
  {-x ≥ 0}
    y = -x;
  {y ≥ 0}
} else {
  {true ∧ ¬(x < 0)}
  {x ≥ 0}
    y = x;
  {y ≥ 0}
}
```

2. Implikationen zeigen:
   1. $true \wedge (x<0) \implies (-x \geq 0)$ ✓
   2. $true \wedge \neg(x<0) \implies (x \geq 0)$ ✓

(zur Klarheit)

   1. $(x < 0) \implies (-x \geq 0)$ ✓
   2. $(x >= 0) \implies (x \geq 0)$ ✓

Beispiel 2: Ungültiges Tripel

Geändert

Zu zeigen: Gültigkeit von $\{true\}$ `if (x < 0) { y = -x; } else { y = x;}` $\{y \geq 0\}$

1. Regeln anwenden:

```
if (x < 0) {
  {true ∧ x < 0}
  {-x ≥ 0}
    y = -x;
  {y ≥ 0}
} else {
  {true ∧ ¬(x < 0)}
  {x ≥ 0}
    y = x;
  {y ≥ 0}
}
```

2. Implikationen zeigen:
   1. $(x < 0) \implies (-x \geq 0)$
   2. $(x \geq 0) \implies (x \geq 0)$

Beispiel 2: Ungültiges Tripel

Geändert

Zu zeigen: Gültigkeit von $\{true\}$ `if (x < 0) { y = -x; } else { y = x;}` $\{y \geq 0\}$

1. Regeln anwenden:

```
if (x < 0) {
  {true ∧ x < 0}
  {-x ≥ 0}
    y = -x;
  {y ≥ 0}
} else {
  {true ∧ ¬(x < 0)}
  {x ≥ 0}
    y = x;
  {y ≥ 0}
}
```

2. Implikationen zeigen:
   1. $(x < 0) \implies (-x \geq 0)$ ✓
   2. $(x \geq 0) \implies (x \geq 0)$

Beispiel 2: Ungültiges Tripel

Geändert

Zu zeigen: Gültigkeit von $\{true\}$ `if (x < 0) { y = -x; } else { y = x;}` $\{y \geq 0\}$

1. Regeln anwenden:

```
if (x < 0) {
  {true ∧ x < 0}
  {-x ≥ 0}
    y = -x;
  {y ≥ 0}
} else {
  {true ∧ ¬(x < 0)}
  {x ≥ 0}
    y = x;
  {y ≥ 0}
}
```

2. Implikationen zeigen:
   1. $(x < 0) \Longrightarrow (-x \geq 0)$ ✓
   2. $(x \geq 0) \Longrightarrow (x \geq 0)$ ✗

Hält nicht mehr
(Gegenbeispiel: $x == 0$)

Beispiel 3: Ohne else-Zweig

Leerer else-Block entspricht fehlendem else-Block

Beispiel

```
{x == y ∧ x != 0}
    if (a > 1) {
        a = a * x;
    }
{a != y}
```

Beispiel 3: Ohne else-Zweig

Leerer else-Block entspricht fehlendem else-Block

Beispiel

```
{x == y ∧ x != 0}
    if (a > 1) {
        a = a * x;
    }
{a != y}
```

```
{x == y ∧ x != 0}
    if (a > 1) {
        a = a * x;
    } else {


    }
{a != y}
```

Beispiel 3: Ohne else-Zweig

```
{x == y ∧ x != 0}
    if (a > 1) {
        a = a * x;
    }
{a != y}
```

1. Regeln anwenden:

```
if (a > 1) {
  {(x == y ∧ x != 0) ∧ a > 1}
  {a * x ≠ y}
    a = a * x;
  {a ≠ y}
} else {
  {(x == y ∧ x != 0) ∧ ¬(a > 1)}

  {a ≠ y}
  {a ≠ y}
}
```

2. Implikationen zeigen:
   1. $(x == y \wedge x != 0 \wedge a > 1) \implies (a * x != y)$
   2. $(x == y \wedge x != 0 \wedge a \leq 1) \implies (a != y)$

Beispiel 3: Ohne else-Zweig

```
{x == y ∧ x != 0}
    if (a > 1) {
        a = a * x;
    }
{a != y}
```

1. Regeln anwenden:

```
if (a > 1) {
  {(x == y ∧ x != 0) ∧ a > 1}
  {a * x ≠ y}
    a = a * x;
  {a ≠ y}
} else {
  {(x == y ∧ x != 0) ∧ ¬(a > 1)}

  {a ≠ y}
  {a ≠ y}
}
```

2. Implikationen zeigen:
   1. ✓ $(x == y \land x != 0 \land a > 1) \implies (a * x != y)$
   2. ✗ $(x == y \land x != 0 \land a \leq 1) \implies (a != y)$

Gültigkeit eines Triples prüfen

```
1// no Overflow, all int
2
3{ x > y }
4
5x = 2 * y;
6
7{ x > y }
```

```
{x > 0}

y = x * x;
z = y / 2;

{z > 0}
```

Gültigkeit eines Triples prüfen

```
1// no Overflow, all int
2
3{ x > y }
4
5x = 2 * y;
6
7{ x > y }
```

Ungültig, vgl.
negative Zahlen

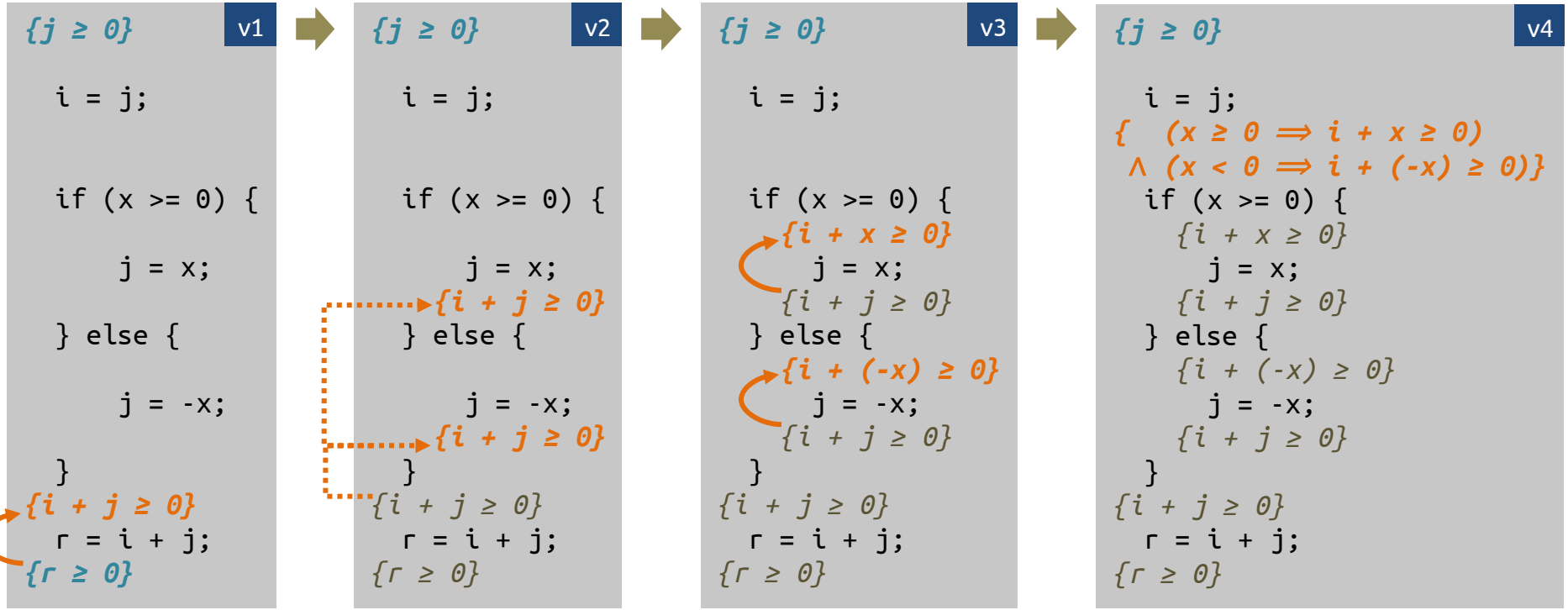
```
{x > 0}

y = x * x;
z = y / 2;

{z > 0}
```

Ungültig, vgl. x = 1,
integer division

Beispiel Schritt für Schritt



Beispiel Schritt für Schritt

v4 ⇒ v5

```
{j ≥ 0}
{  (x ≥ 0 ⟹ j + x ≥ 0)
 ∧ (x < 0 ⟹ j + (-x) ≥ 0) }
  i = j;
{  (x ≥ 0 ⟹ i + x ≥ 0)
 ∧ (x < 0 ⟹ i + (-x) ≥ 0) }
  if (x >= 0) {
    {i + x ≥ 0}
      j = x;
    {i + j ≥ 0}
  } else {
    {i + (-x) ≥ 0}
      j = -x;
    {i + j ≥ 0}
  }
{i + j ≥ 0}
  r = i + j;
{r ≥ 0}
```

Dann noch zu zeigen:

$$(j \geq 0) \Longrightarrow (\ (x \geq 0 \Longrightarrow j + x \geq 0) \wedge (x < 0 \Longrightarrow j + (-x) \geq 0))$$

Fallunterscheidung (zwei Konjunkte):

1. Wenn $j \geq 0$ und $x \geq 0$, dann $j + x \geq 0$ ✓
2. Wenn $j \geq 0$ und $x < 0$, dann $j + (-x) \geq 0$ ✓

Hoare Triple
Übungsaufgaben zu gültig/ungültig

Hoare Tripel – Prüfungsbeispiele HS18

```
{ b > c }
 if (x > b) {
        a = x;
} else {
        a = b;
}
{ a > c }
```

```
{ true }
 if (x > y) {
        y = x;
} else {
        y = -x;
}
{ y >= x}
```

Hoare Tripel – Prüfungsbeispiele HS18

```
{ b > c }
 if (x > b) {
        a = x;
} else {
        a = b;
}
{ a > c }
```

$(b > c) \wedge (x > b) \overset{?}{\Rightarrow} x > c$ ✓

$(b > c) \wedge (x \leq b) \overset{?}{\Rightarrow} b > c$ ✓

```
{ true }
 if (x > y) {
        y = x;
} else {
        y = -x;
}
{ y >= x}
```

$true \wedge (x > y) \overset{?}{\Rightarrow} x \geq x$ ✓

$true \wedge (x \leq y) \overset{?}{\Rightarrow} -x \geq x$ ✗

counterexample

$x = 1$,
$y = 2$

Hoare Logik FAQ

- Müssen wir unsere Ausdrücke vereinfachen?
  - Solange die Lösung **logisch äquivalent** ist, ist sie korrekt.

- Müssen wir unsere Herleitung zeigen?
  - Solange dies **nicht explizit gefragt** wird, ist dies nicht nötig.

https://create.kahoot.it/details/u7-kahoot/f981d498-3841-42d6-98d4-d6004503b84c?drawer=

Aufgabe 1: Präfixkonstruktion

Gegeben seien zwei Strings `s` und `t` und ein Integer `n` mit $n \geq 0$. Schreiben Sie ein Programm, das zurückgibt, ob `s` eine Konkatenation von maximal `n` vielen Präfixen von `t` ist.
Beispiele:

- `s = "abcababc"`, `t = "abc"`, `n = 4`: Das Programm sollte `true` zurückgeben, da `"abc"` und `"ab"` Präfixe von `t` sind und `s` eine Konkatenation von `"abc"`, `"ab"`, `"abc"` ist.
- `s = "abcbcabc"`, `t = "abc"`, `n = 4`: Das Programm sollte `false` zurückgeben, da `"bc"` kein Präfix von `t` ist.
- `s = "abab"`, `t = "abac"`, `n = 2`: Das Programm sollte `true` zurückgeben, da `"ab"` ein Präfix von `t` ist und `s` eine Konkatenation von `"ab"`, `"ab"` ist.

Implementieren Sie die Methode `isPrefixConstruction(String s, String t, int n)` in der Klasse `PrefixConstruction`. Die Methode hat drei Argumente: die beiden Strings `s` und `t` und der Integer `n`. Sie dürfen davon ausgehen, dass der Integer grösser oder gleich 0 ist. In der Datei "PrefixConstructionTest.java" finden Sie Tests.

Tipp: Lösen Sie die Aufgabe rekursiv.

Aufgabe 1: Präfixkonstruktion

- prüfen, ob sich ein Ziel-String **s** aus maximal **n** Bausteinen zusammensetzen lässt, wobei jeder dieser Bausteine ein Präfix (Anfangsteil) des Strings **t** sein muss
- z.B. **t = abc**. Also sind "a", "ab" und "abc" mögliche Bausteine
- gehe von links nach rechts durch und prüfe für alle möglichen Präfix-Längen (Bausteine), ob der String damit beginnt. Wenn ja, entferne den Teil, und rufe rekursiv auf den Rest auf

Aufgabe 1: Präfixkonstruktion

- **Beispiel**

s=“ababa”, t=“aba”, n=2

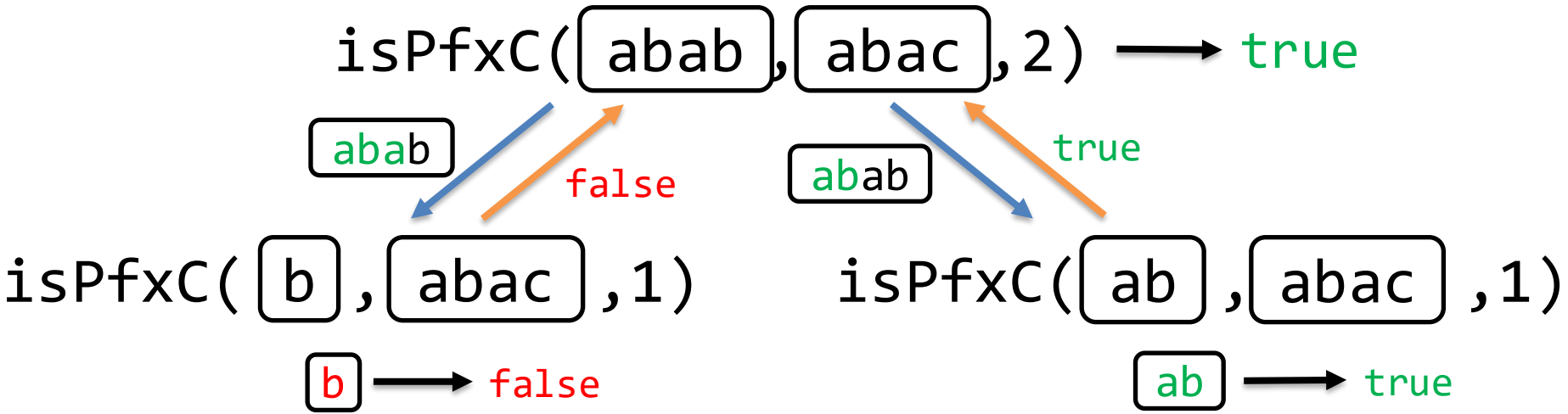
Neues Beispiel

1. **isPrefix("ababa", "aba", 2)**

   Prüfe, welche Präfixe passen würden: "a", "ab" und "aba"

2. **Für jede dieser Präfixe wird ein rekursiver Aufruf gestartet**
   1. «a»: isPrefix("baba", "aba", 1) -> false
   2. «ab»: isPrefix("aba", "aba", 1) -> true
   3. «aba»: isPrefix("ba", "aba", 1) -> false
3. **Falls irgendwo true, return true**

Aufgabe 1: Präfixkonstruktion



Towers of Hanoi

Das **Türme von Hanoi-Problem** besteht aus

- drei Stäben
- einer Anzahl von unterschiedlich grossen Scheiben.

Die Scheiben sind zu Beginn auf einem Stab. Die grösste Scheibe ist unten, die kleinste oben.

Ziel: alle Scheiben von dem Ausgangsstab auf den Zielstab bewegen

Dabei: darf immer nur **eine Scheibe auf einmal** bewegt, und **nie eine grössere Scheibe auf eine kleinere gelegt** werden.

Vorgehen bei Rekursiven Problemen

1. Was ist die einfachste mögliche Eingabe?

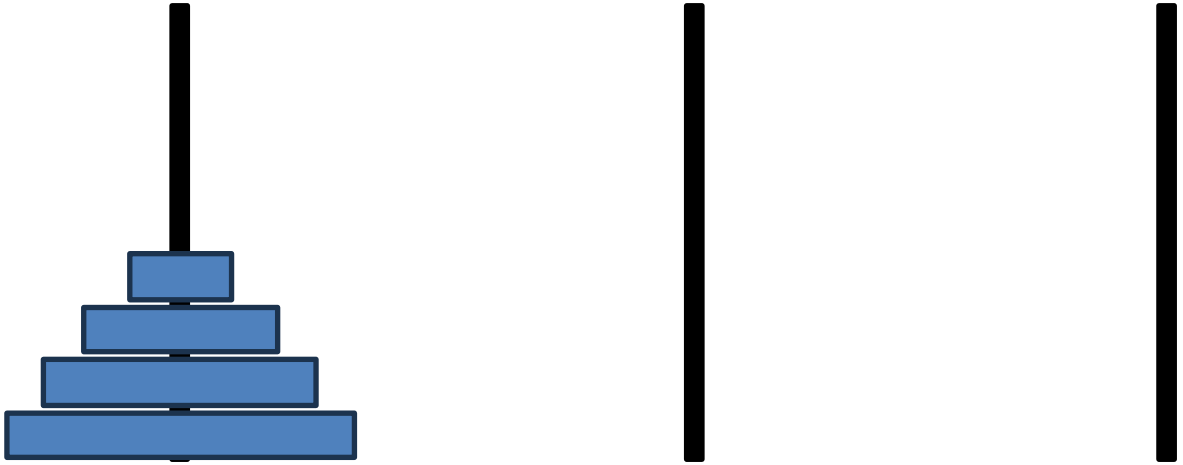
   → Base Case

2. Beispiele ausprobieren und visualisieren
3. Leite grössere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster

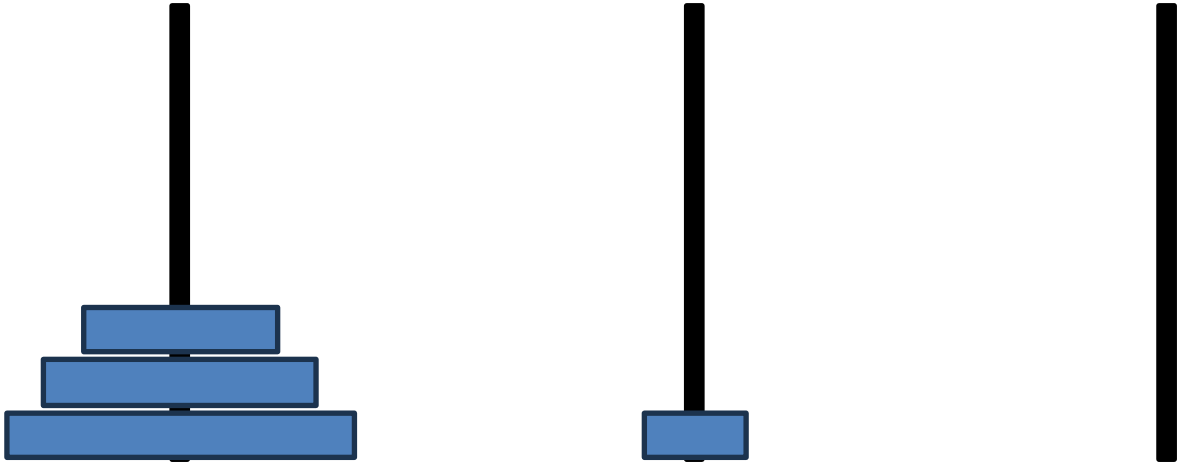
   → Rekursionsschritt

5. Schreibe den Code

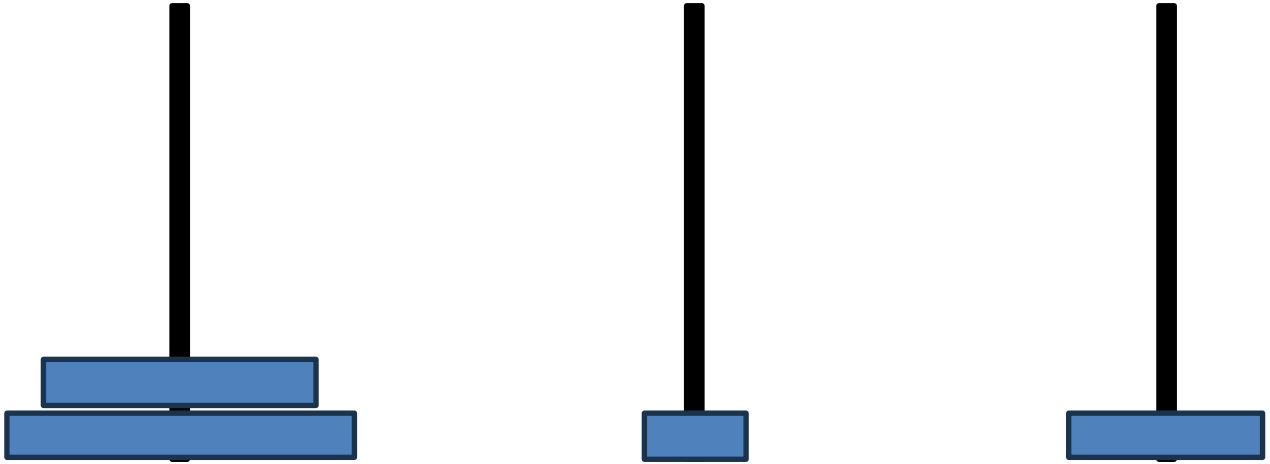
Towers of Hanoi - Beispiel



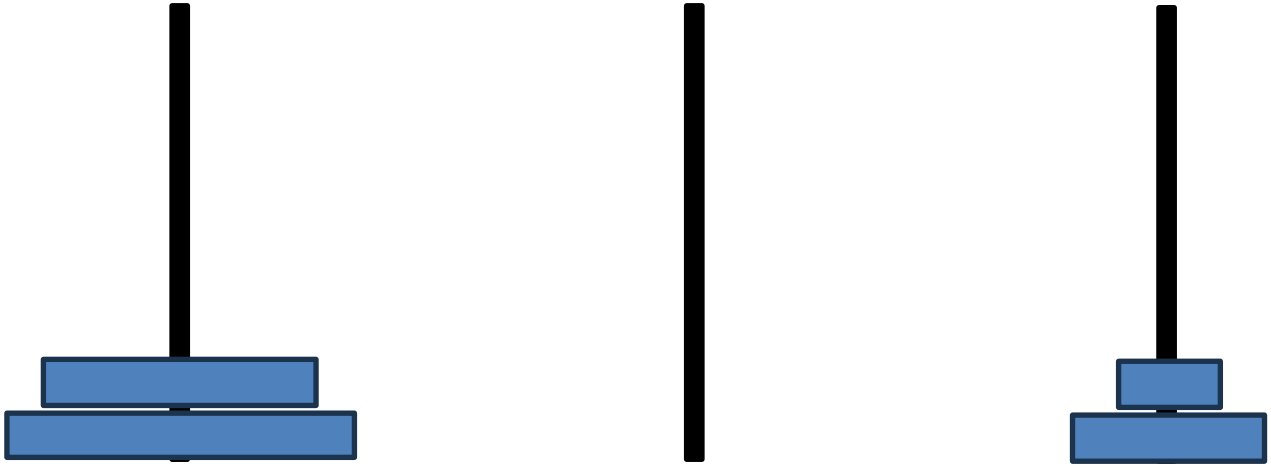
Towers of Hanoi - Beispiel



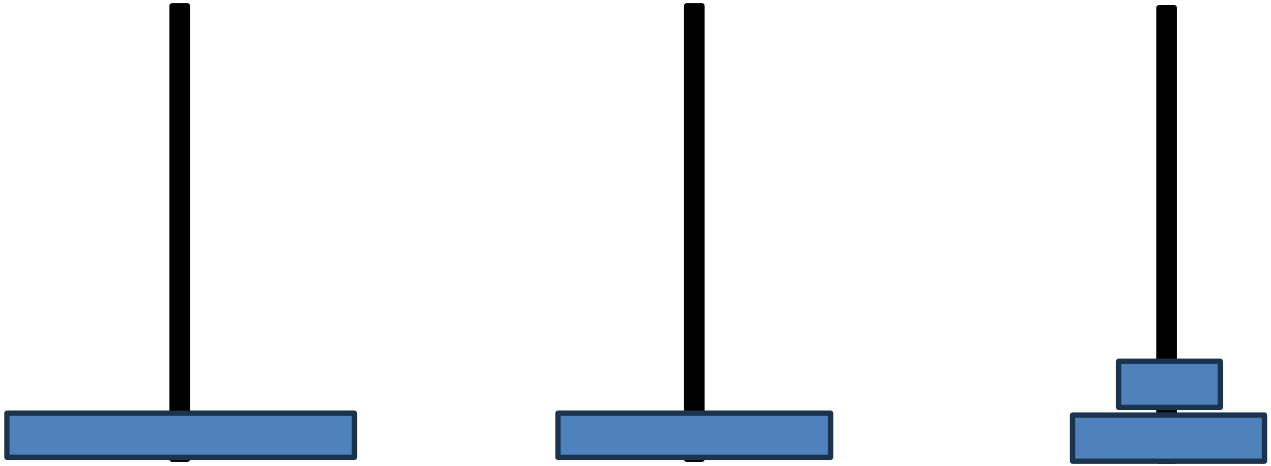
Towers of Hanoi - Beispiel



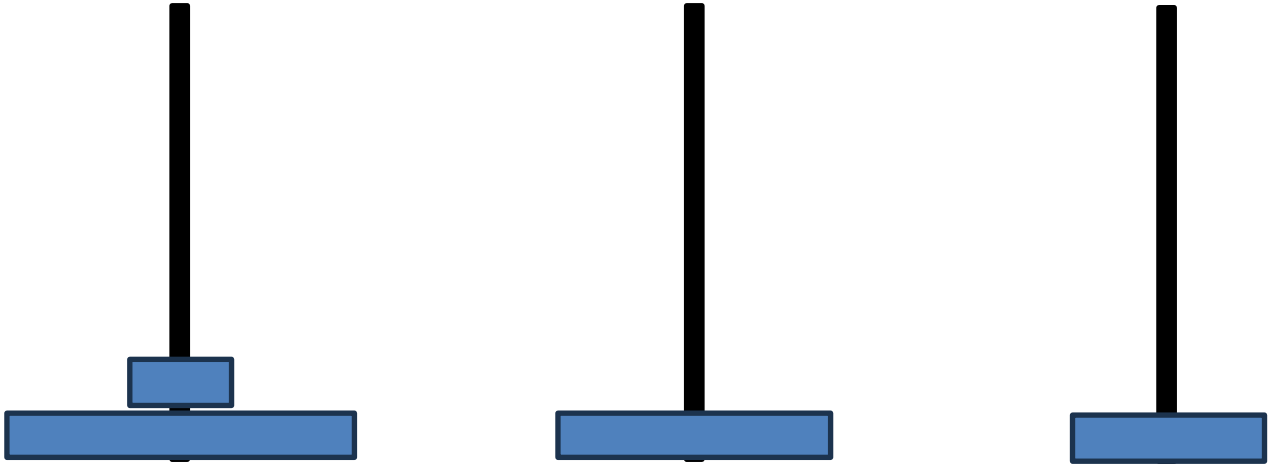
Towers of Hanoi - Beispiel



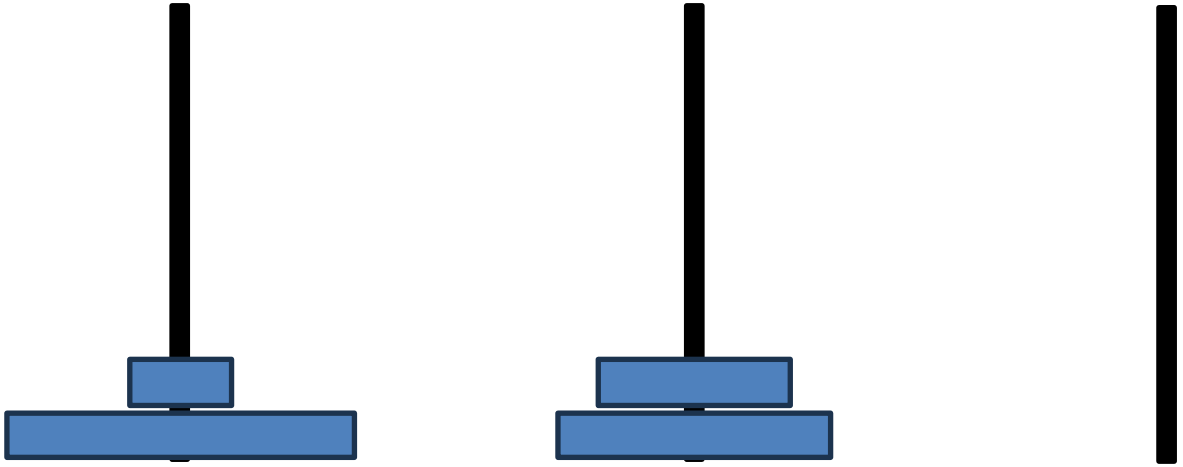
Towers of Hanoi - Beispiel



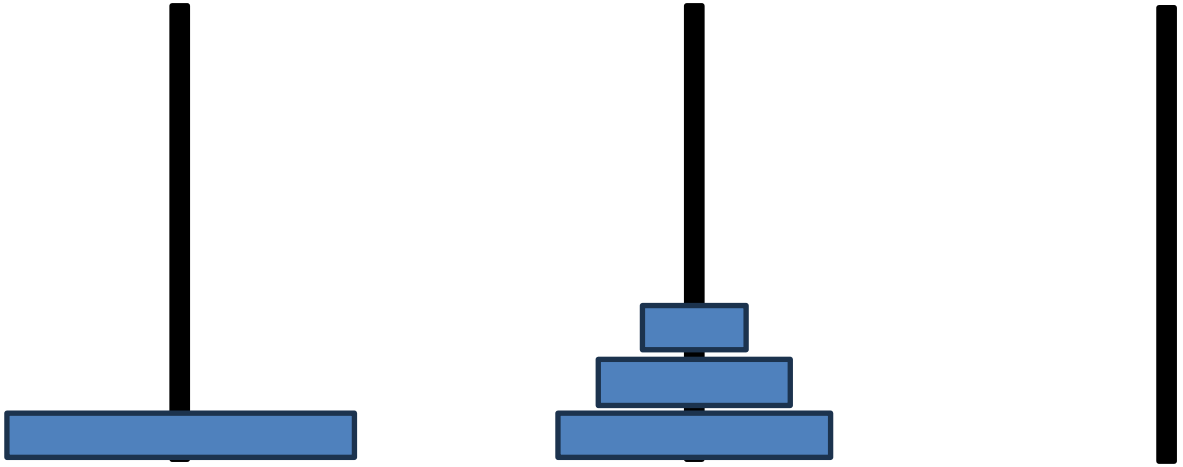
Towers of Hanoi - Beispiel



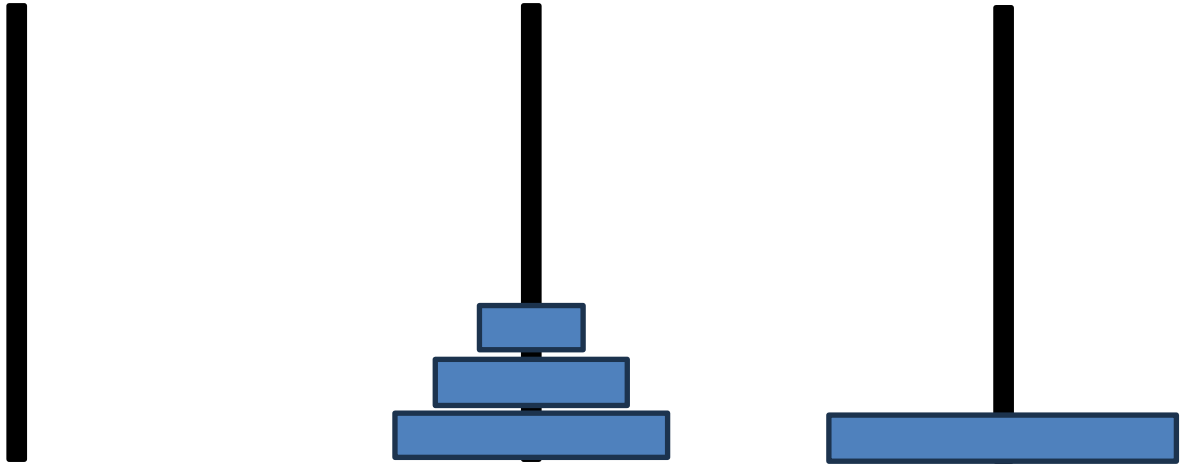
Towers of Hanoi - Beispiel



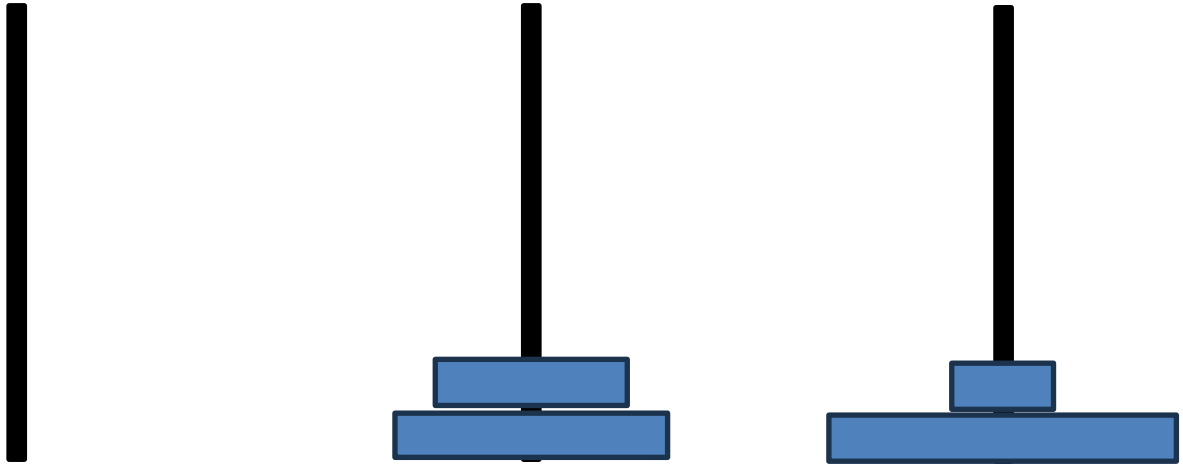
Towers of Hanoi - Beispiel



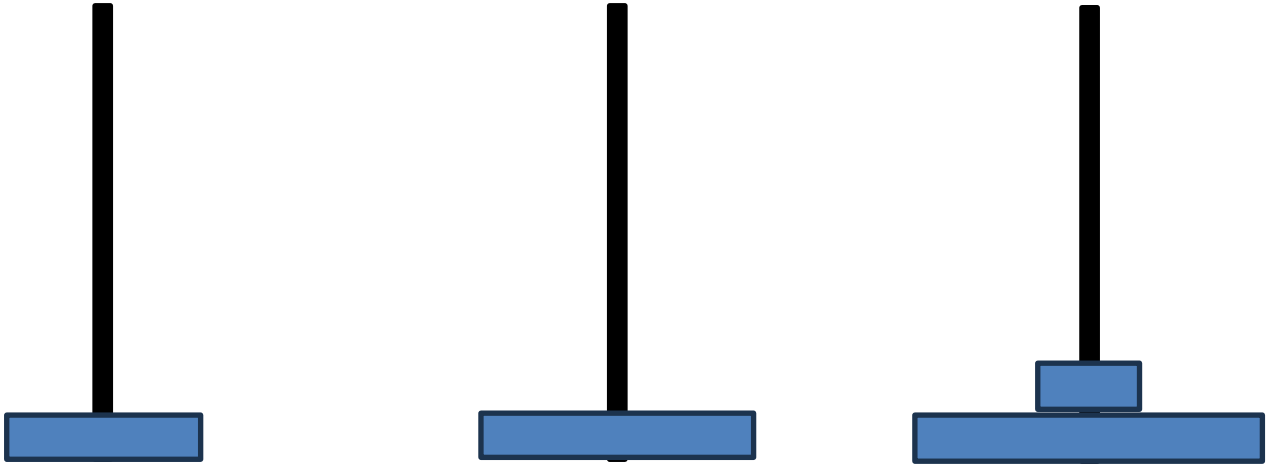
Towers of Hanoi - Beispiel



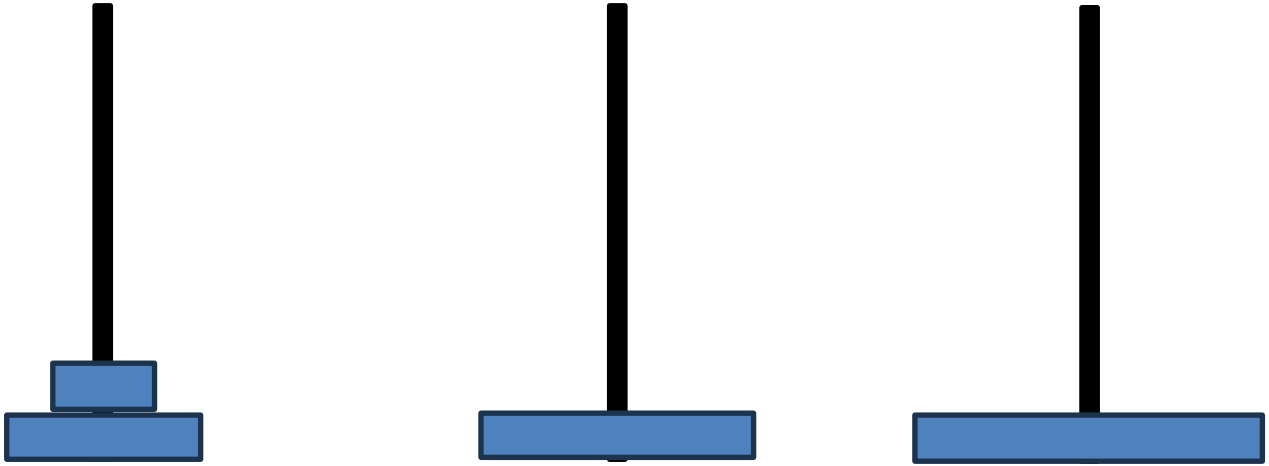
Towers of Hanoi - Beispiel



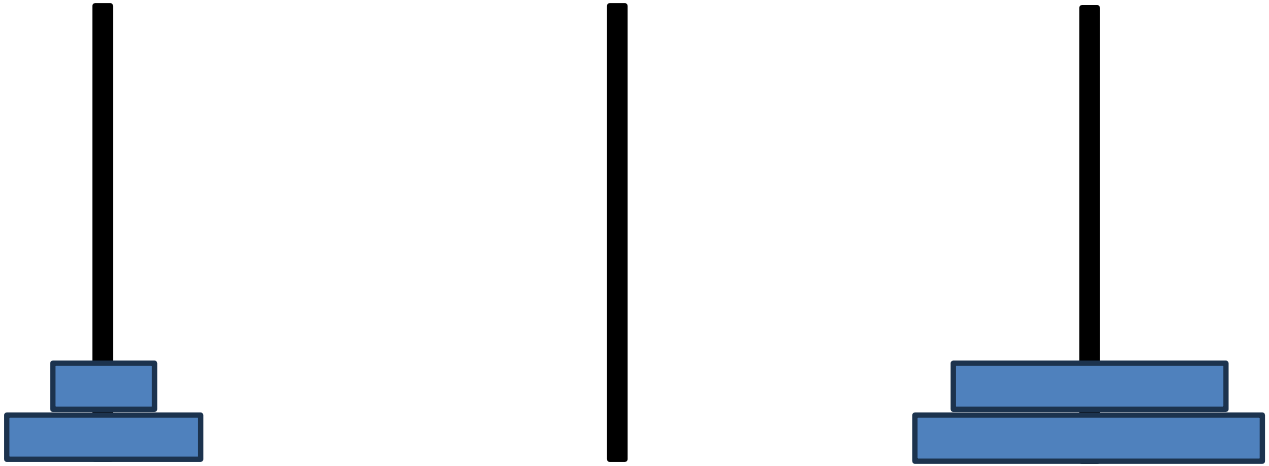
Towers of Hanoi - Beispiel



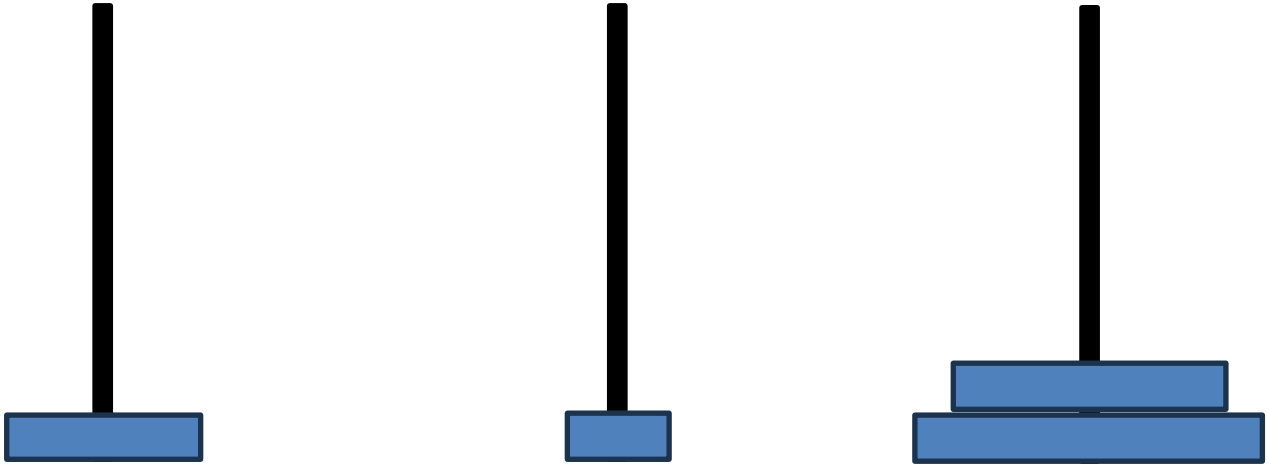
Towers of Hanoi - Beispiel



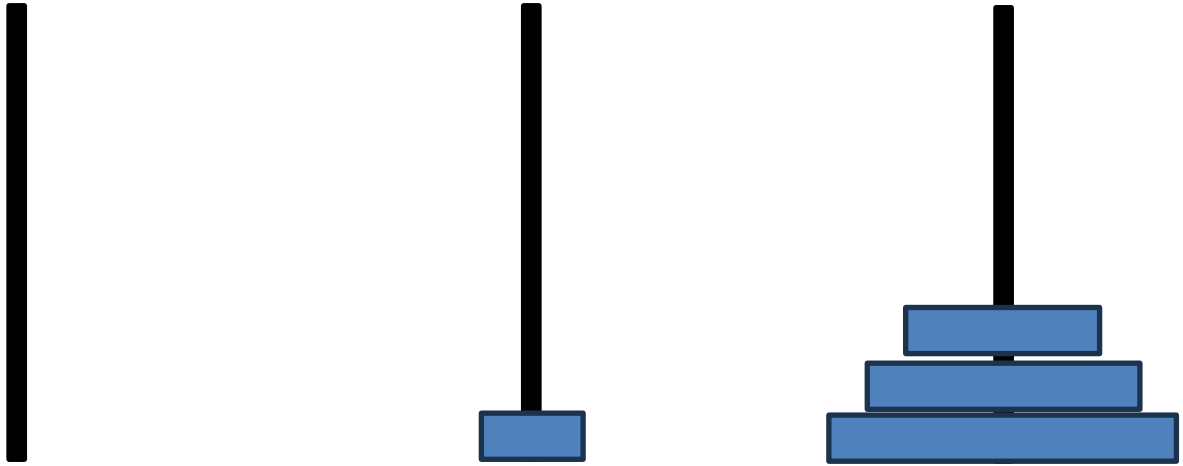
Towers of Hanoi - Beispiel



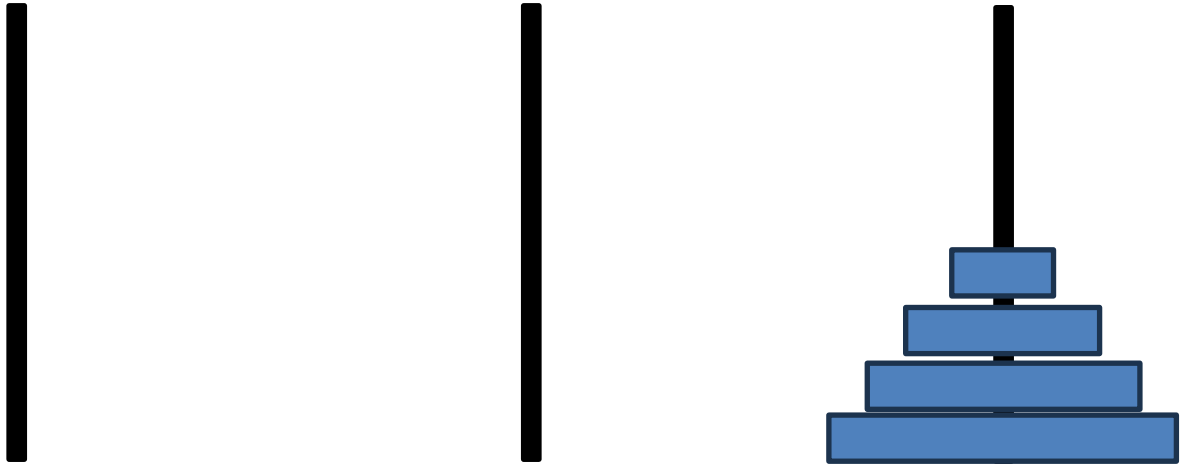
Towers of Hanoi - Beispiel



Towers of Hanoi - Beispiel



Towers of Hanoi - Beispiel

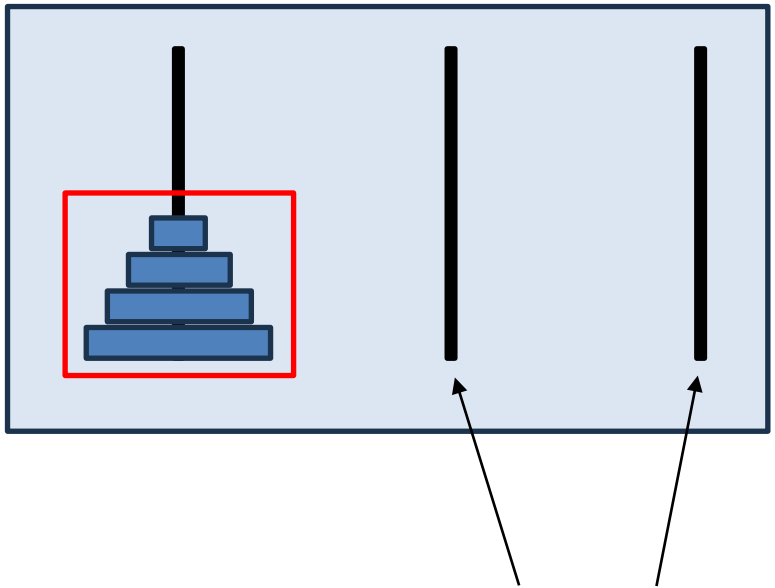


Lösungsidee

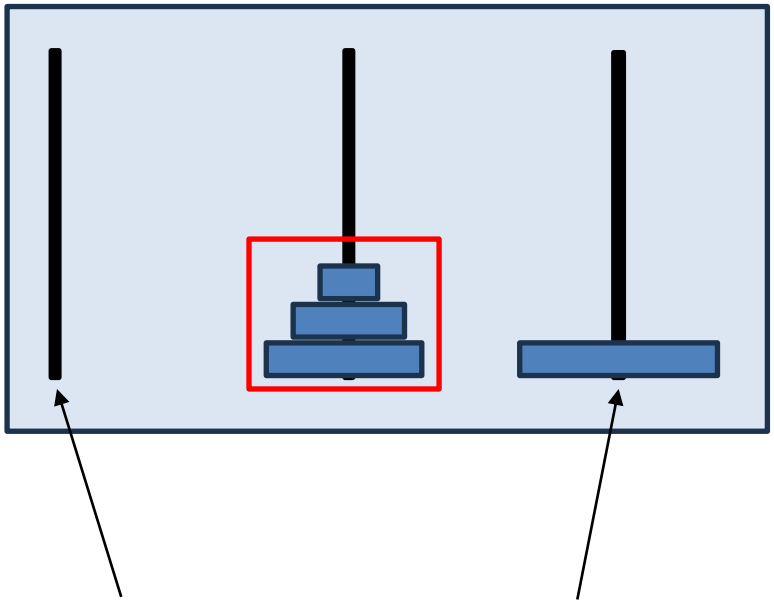
1. **Teilturm auslagern**: Die oberen n-1 Scheiben (rote Box) vom Ausgangsstab auf den Hilfsstab verschieben

2. **Basis verschieben**

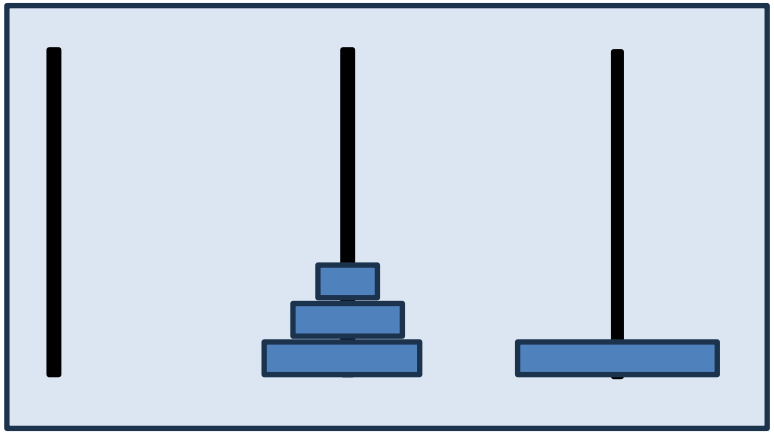
3. **Teilturm ins Ziel bringen**

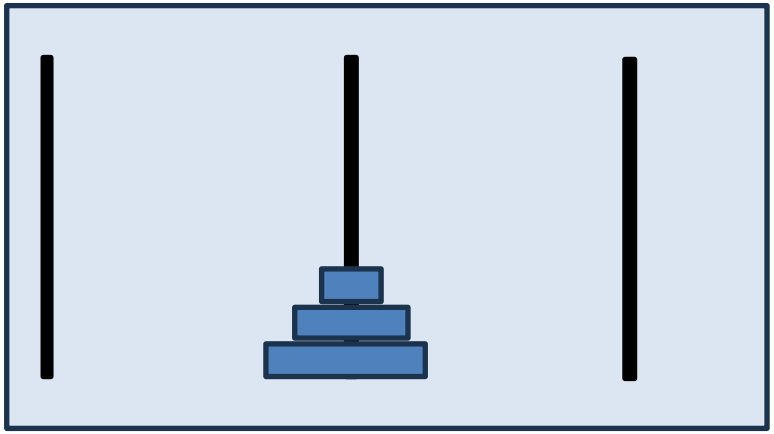


Wir können die Scheiben der roten Box auf den verbleibenden Stäben platzieren.

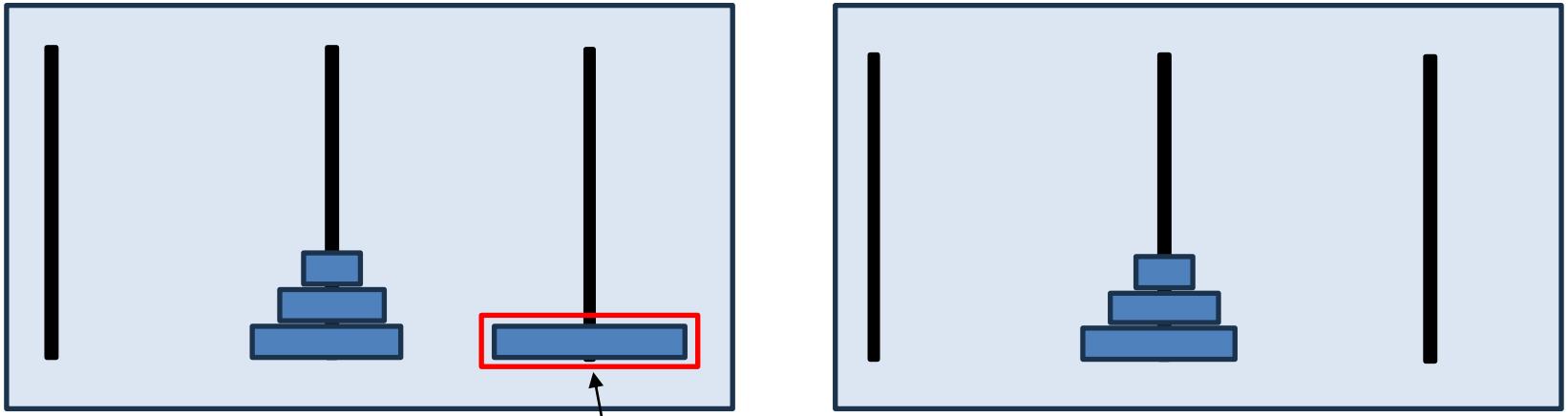


Wir können die Scheiben der roten Box auf den verbleibenden Stäben platzieren.





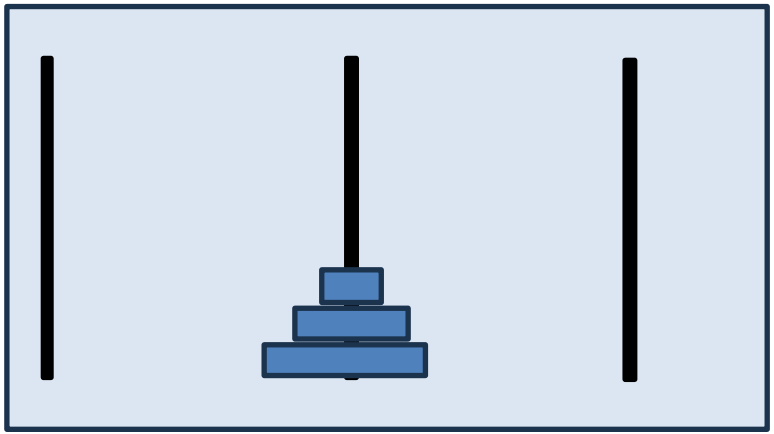
Können die Konstellationen gleich gelöst werden?

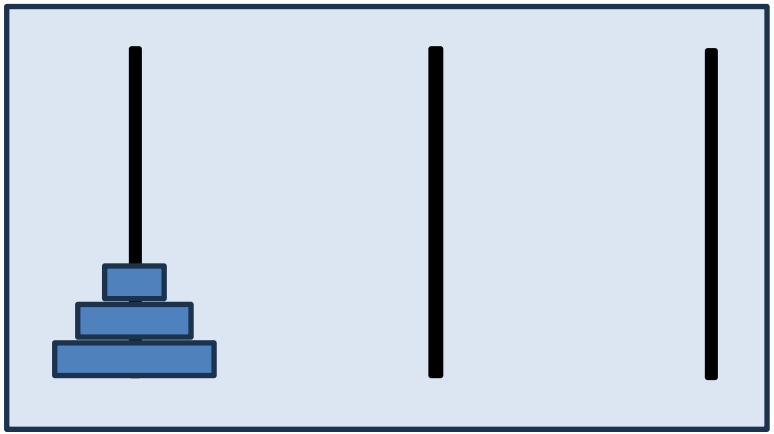


Können die Konstellationen gleich gelöst werden?

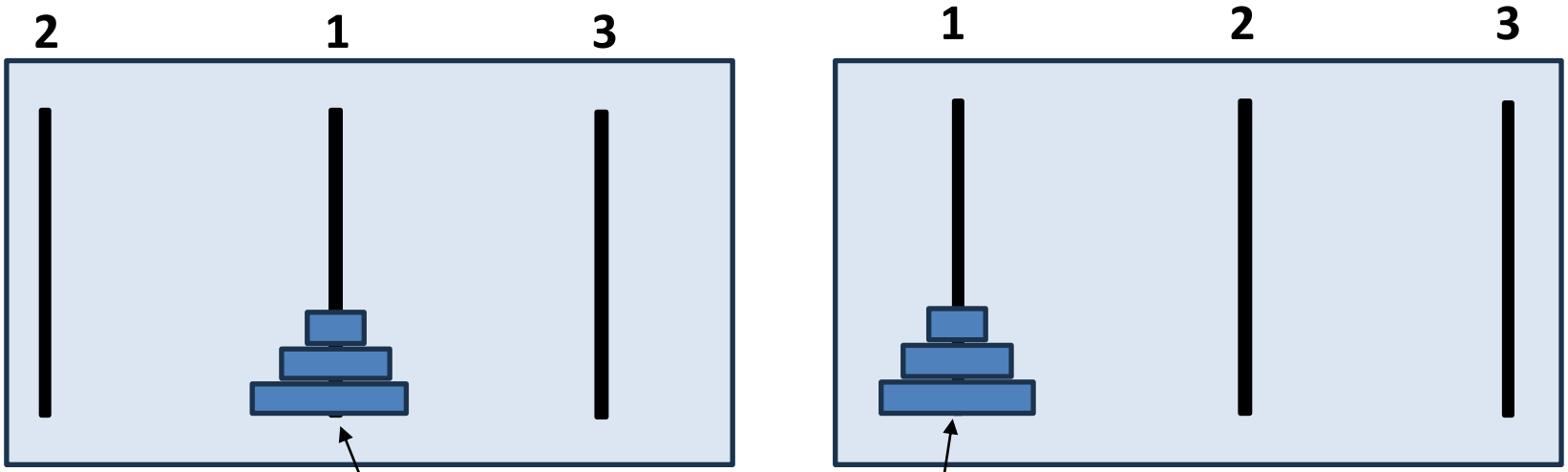
Diese Scheibe ändert das Problem nicht. Es ist die grösste Scheibe und wir können sie deshalb ignorieren und nur die verbleibenden Scheiben betrachten.

Reduktion





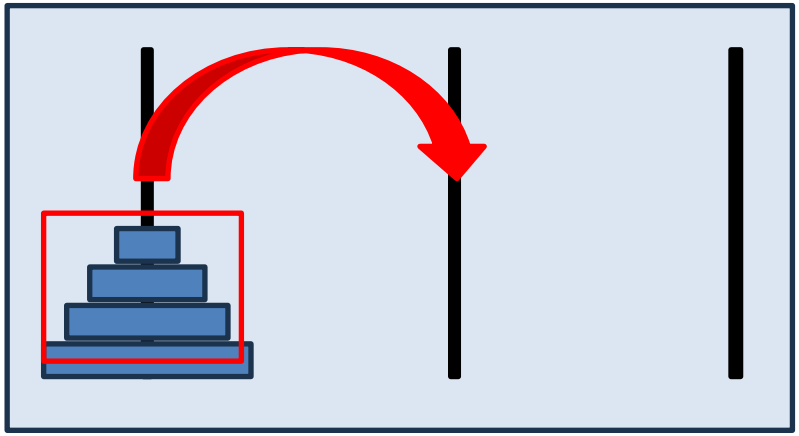
Können die Konstellationen gleich gelöst werden?

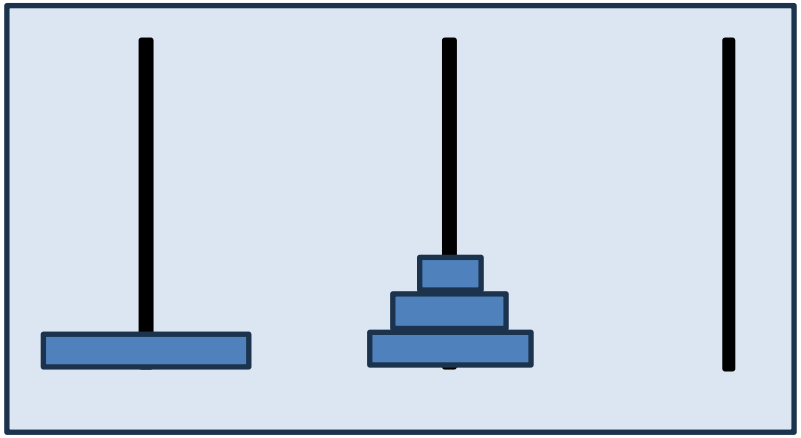


Können die Konstellationen gleich gelöst werden?

Reduktion

Das ist das gleiche Problem, wenn wir die Stäbe anders nummerieren.

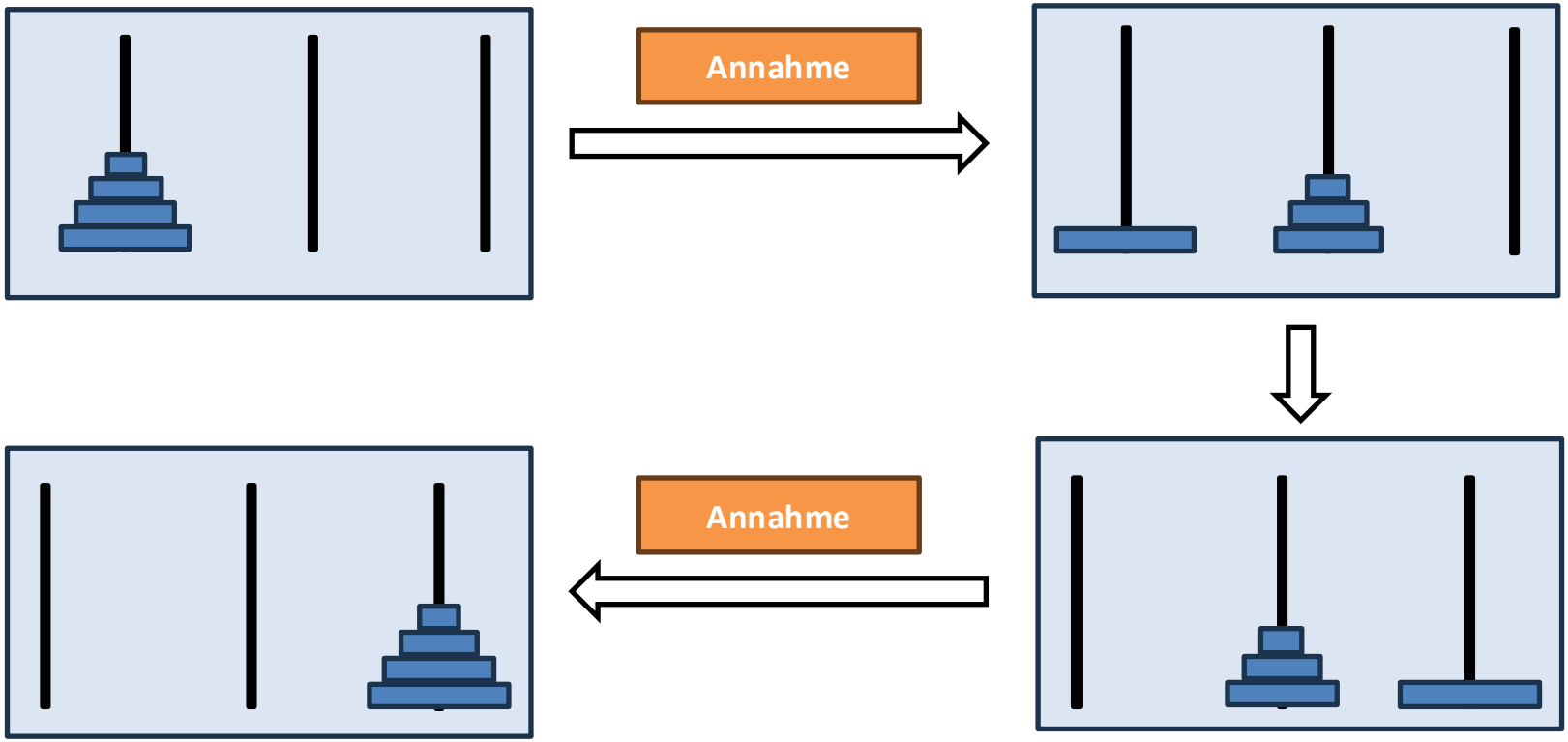




Wir nehmen nun an, wir wissen bereits, wie wir einen Stapel von Grösse drei auf einen “freien” Stab bewegen können, wenn wir zwei “freie” Stäbe haben.

Annahme

Wir können das Problem jetzt lösen!



Lösungsidee

1. **Teilturm auslagern**: Die oberen n-1 Scheiben (rote Box) vom Ausgangsstab auf den Hilfsstab verschieben

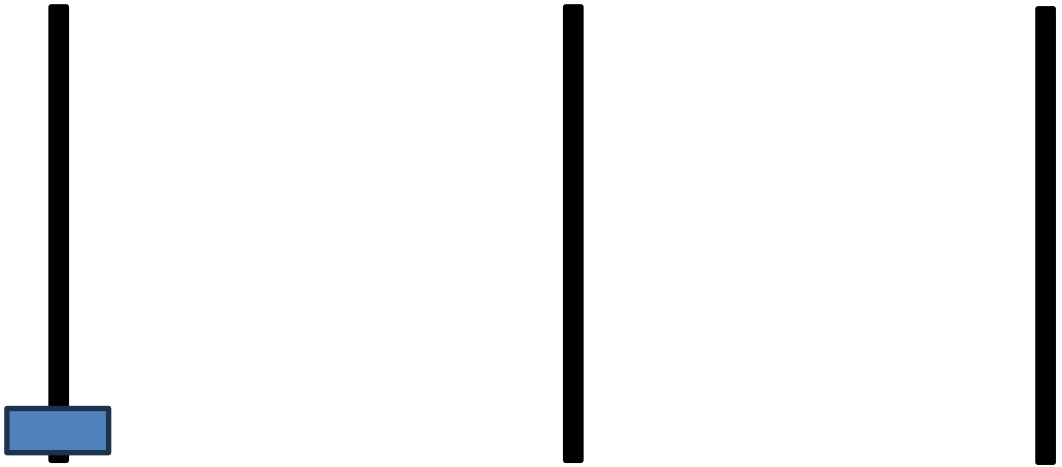
2. **Basis verschieben**

3. **Teilturm ins Ziel bringen**

Stimmt die Annahme?

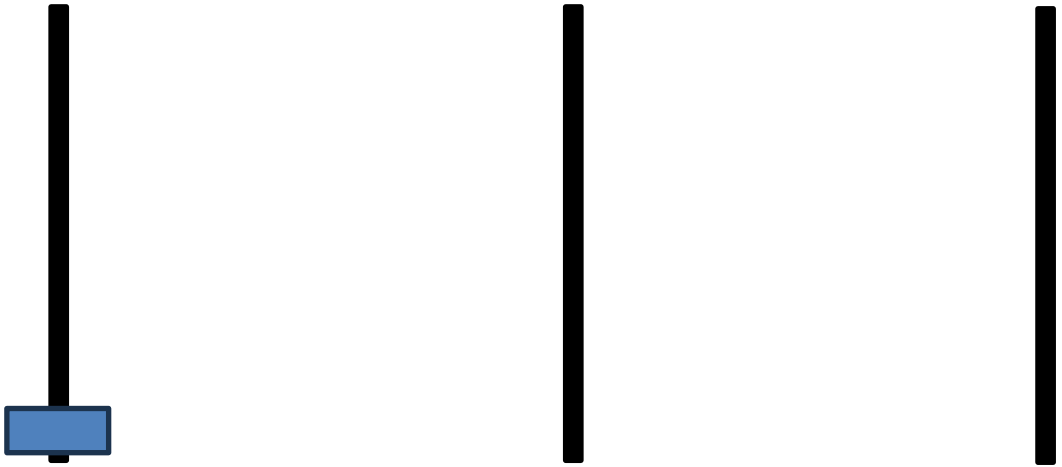
Wir nehmen nun an, wir wissen bereits, wie wir einen Stapel von Grösse drei auf einen “freien” Stab bewegen können, wenn wir zwei “freie” Stäbe haben.

Stimmt die Annahme?



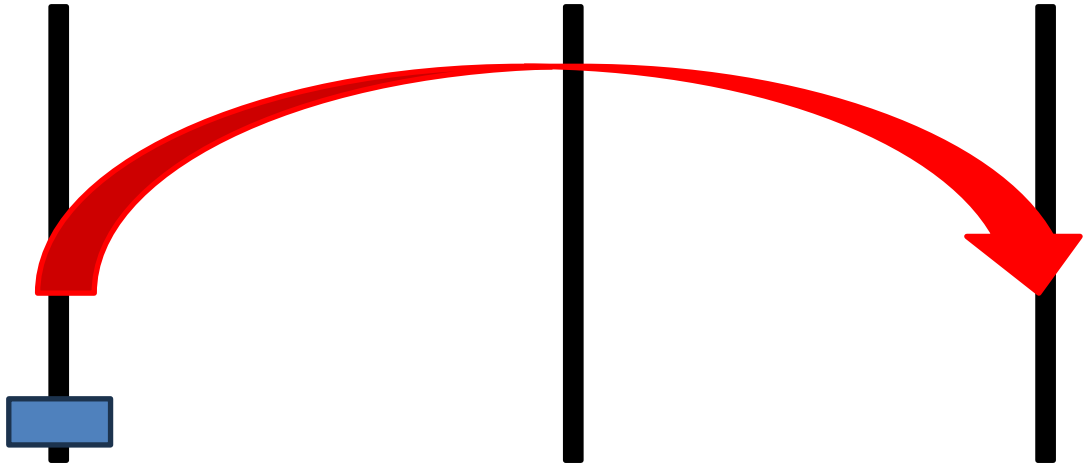
Nur in einem Fall!

Stimmt die Annahme?

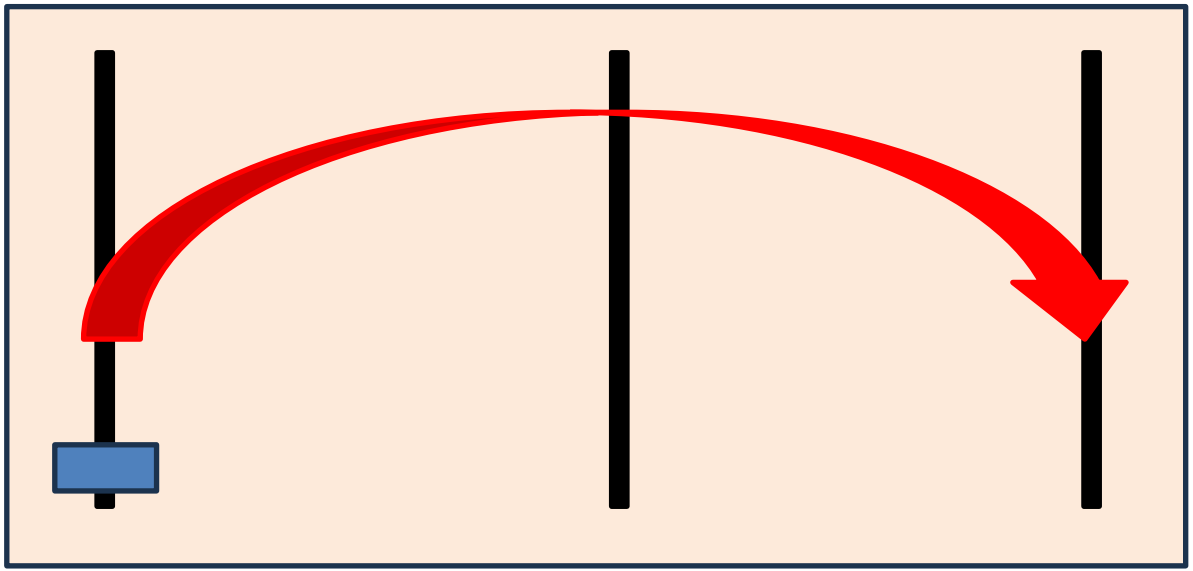


Das reicht aber!

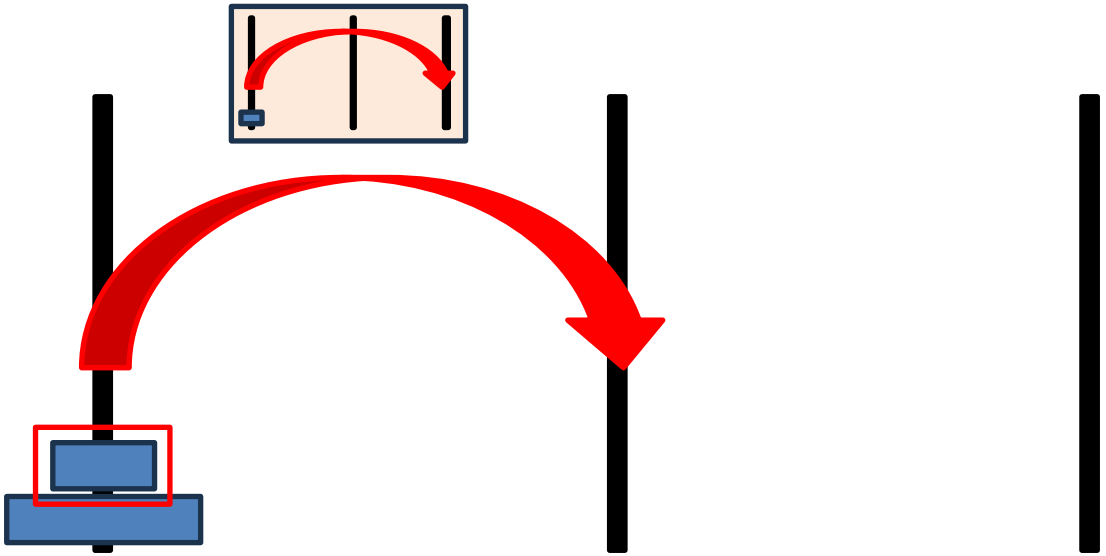
Towers of Hanoi – Step by Step



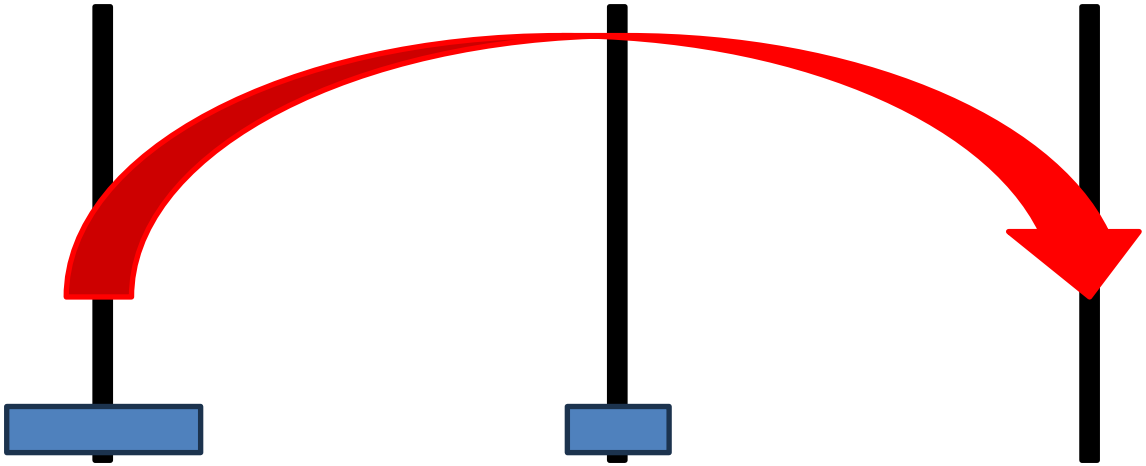
Towers of Hanoi – Step by Step



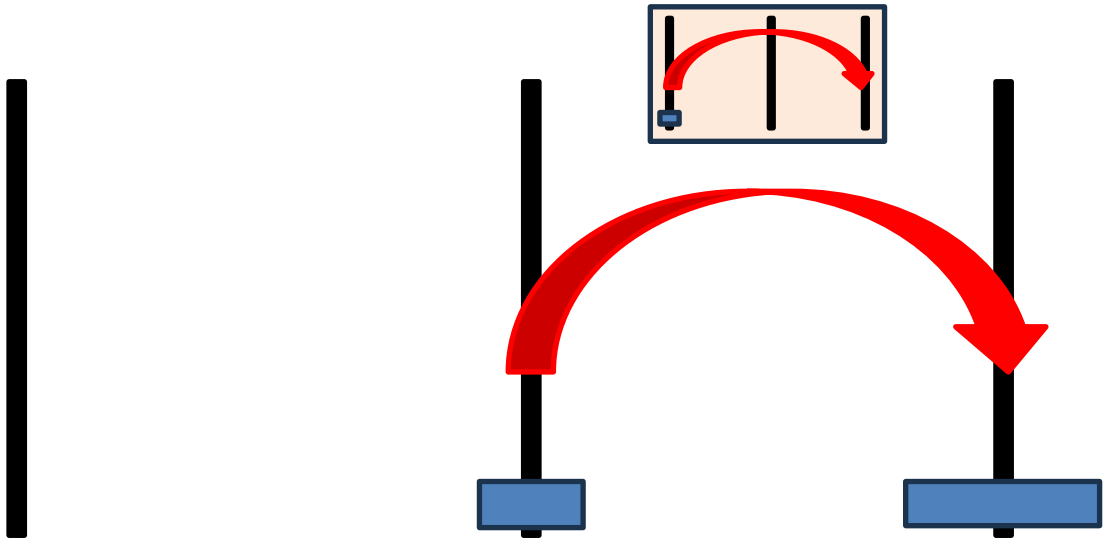
Towers of Hanoi – Step by Step



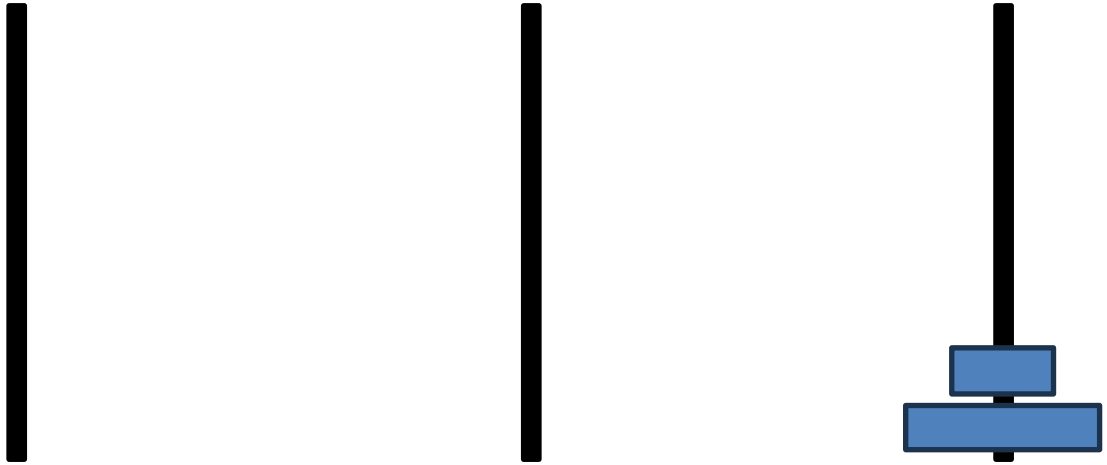
Towers of Hanoi – Step by Step



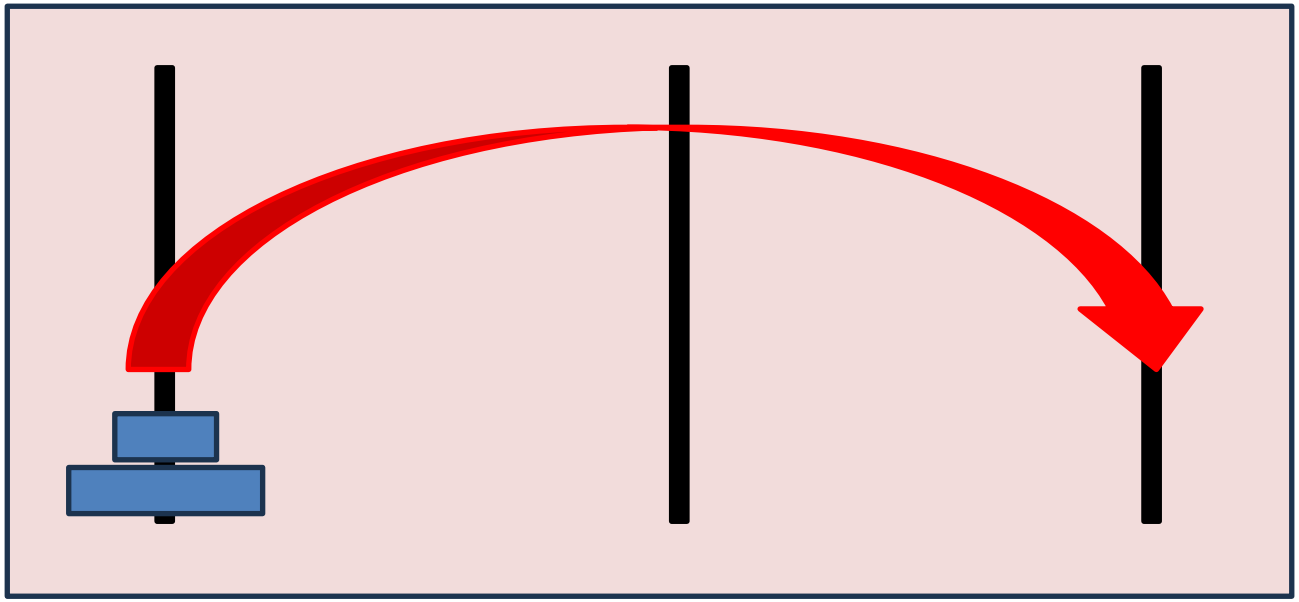
Towers of Hanoi – Step by Step



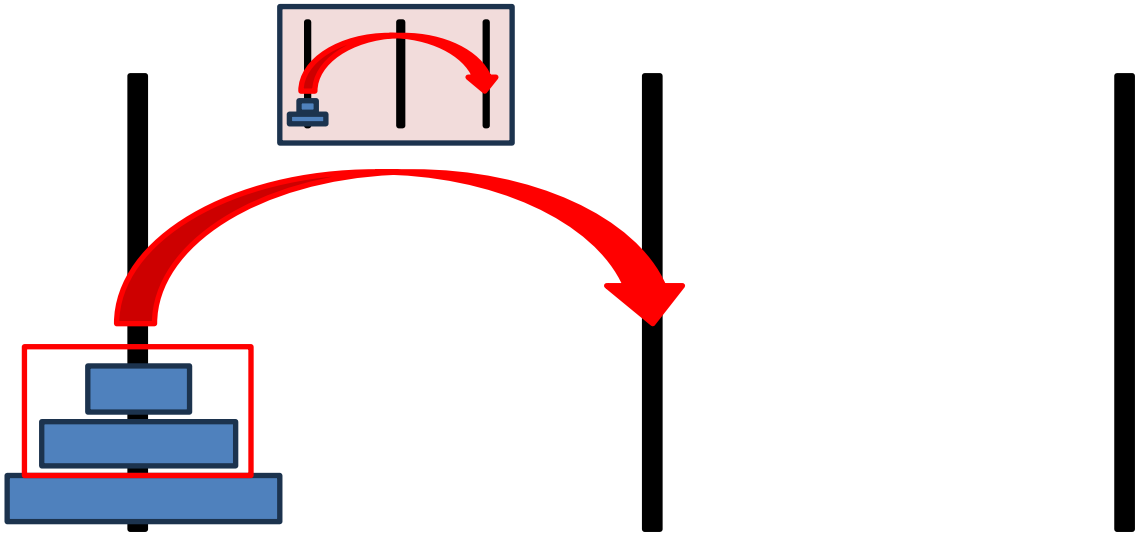
Towers of Hanoi – Step by Step



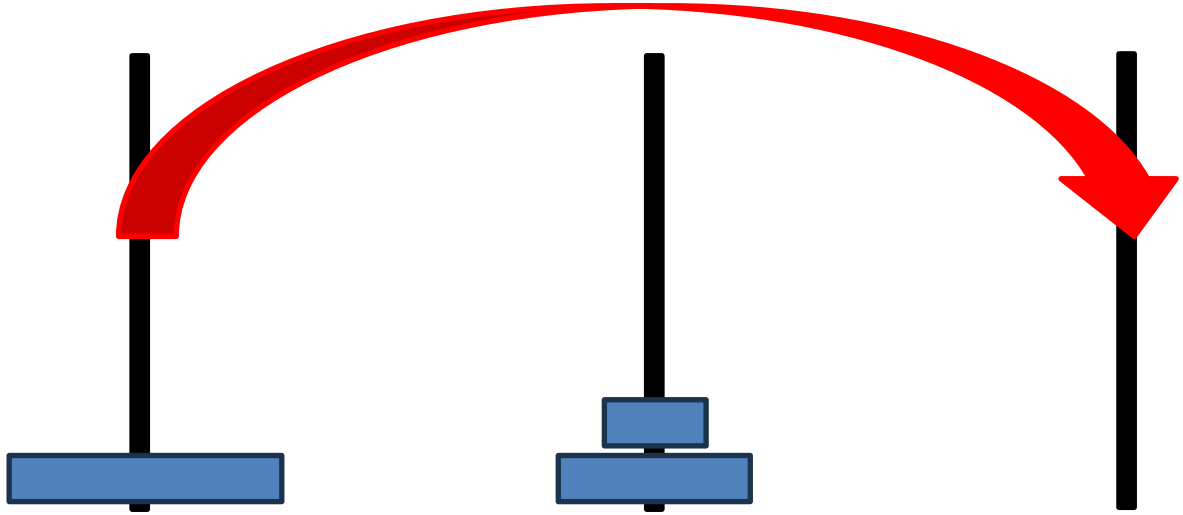
Towers of Hanoi – Step by Step



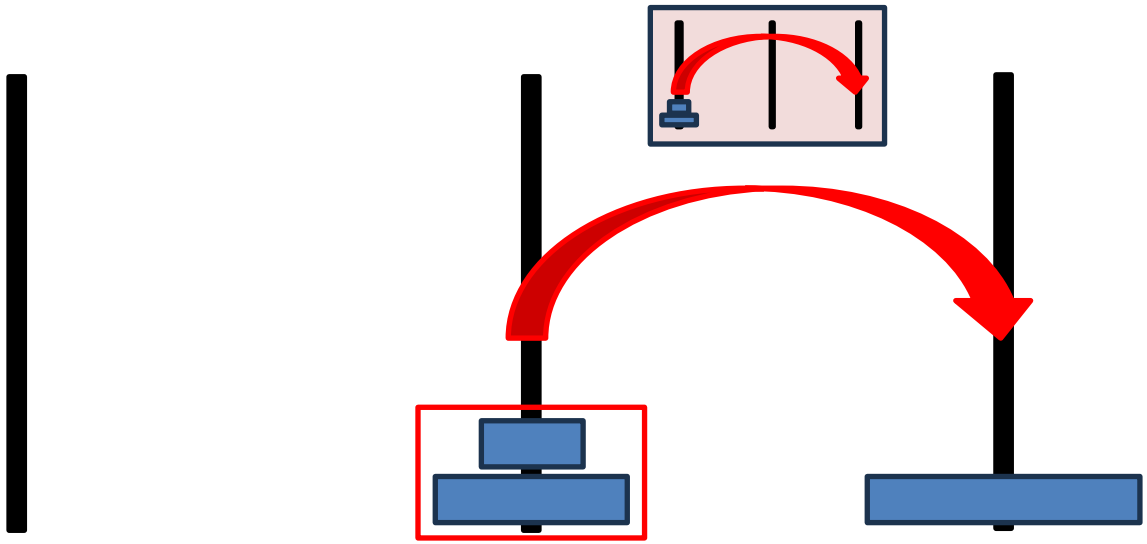
Towers of Hanoi – Step by Step



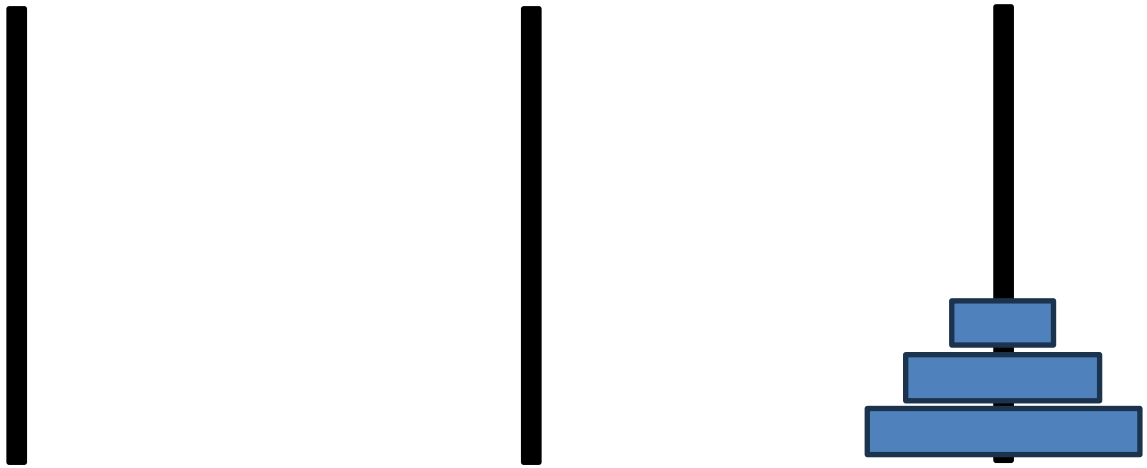
Towers of Hanoi – Step by Step



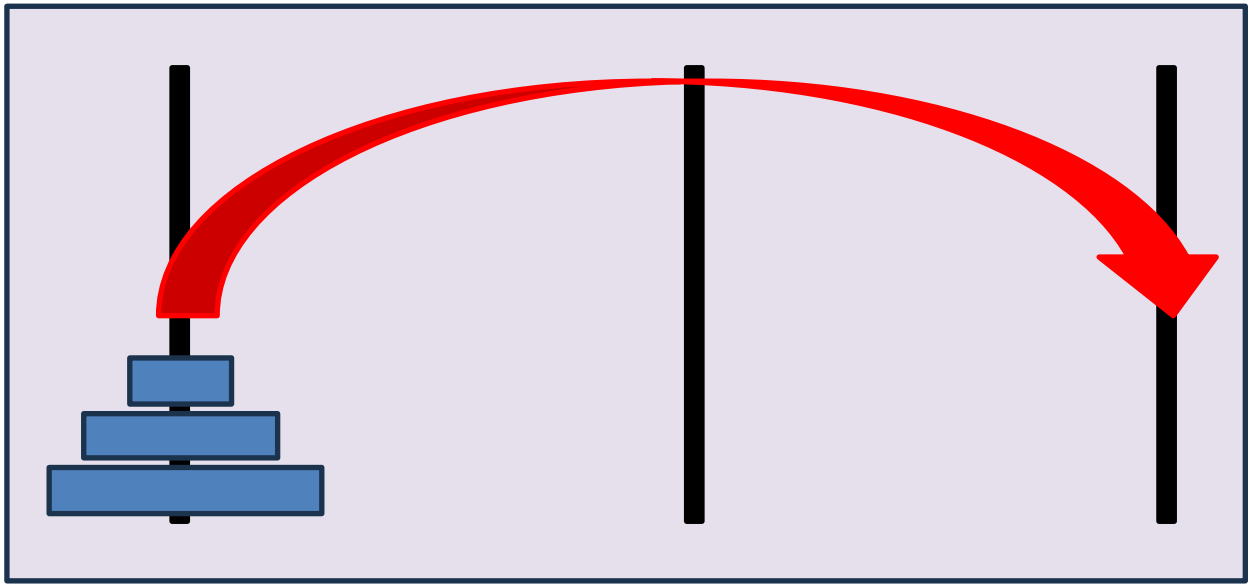
Towers of Hanoi – Step by Step



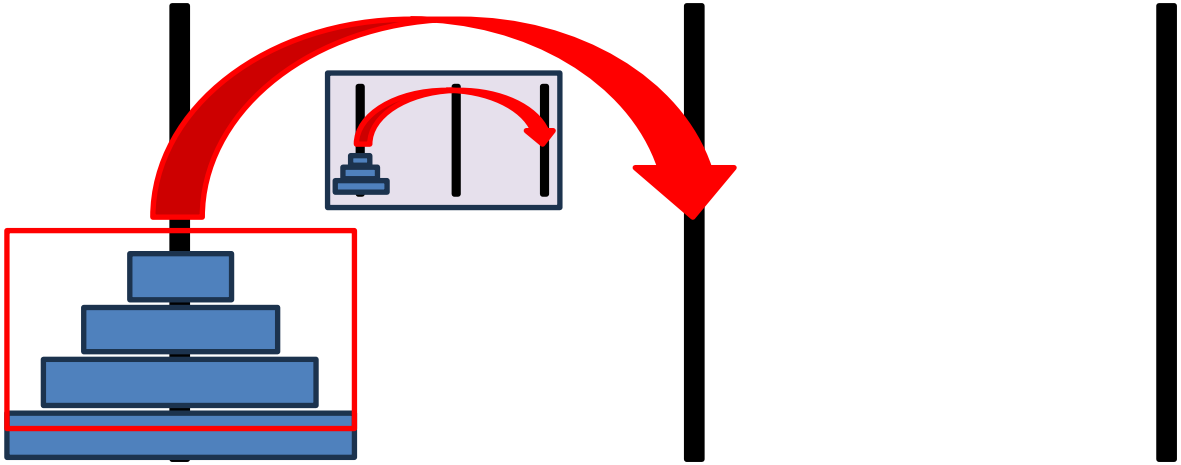
Towers of Hanoi – Step by Step



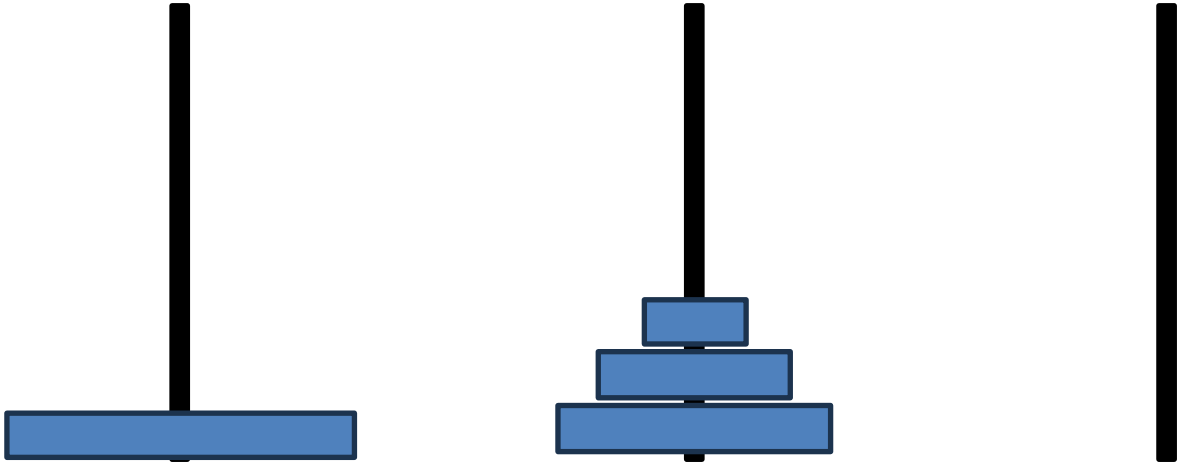
Towers of Hanoi – Step by Step



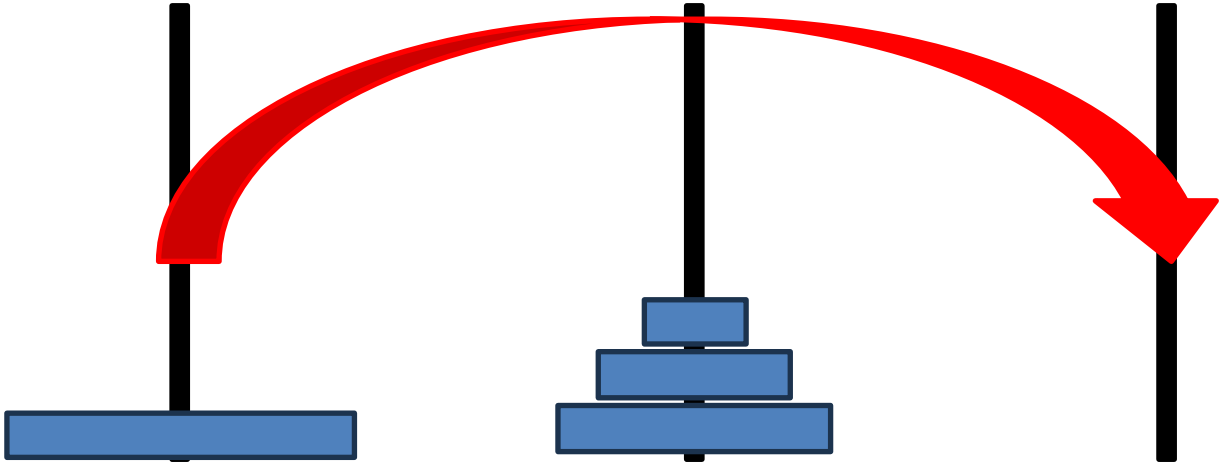
Towers of Hanoi – Step by Step



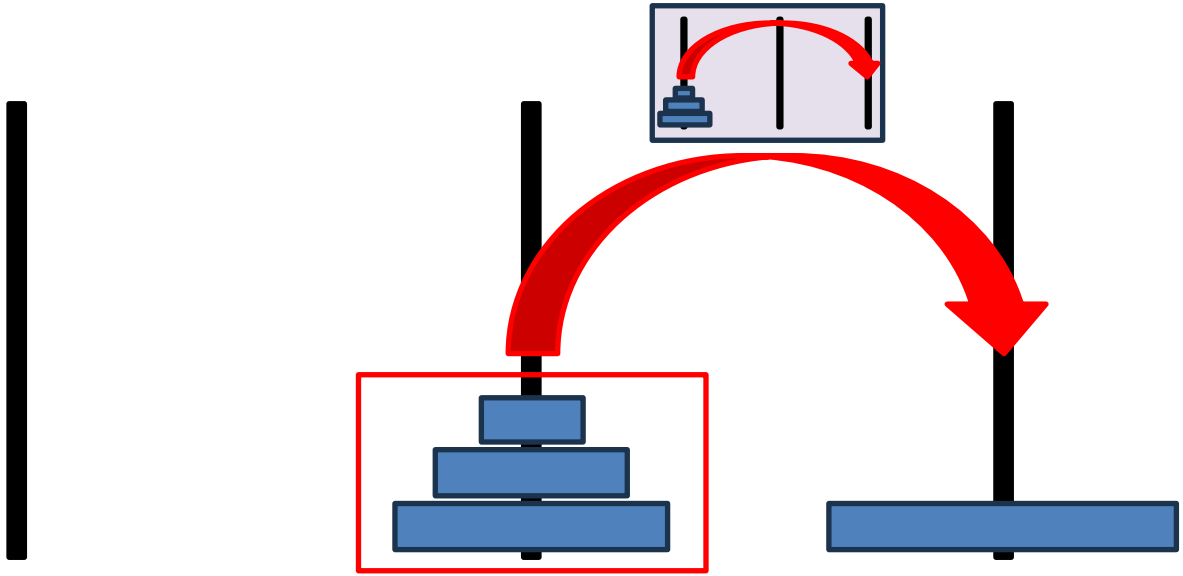
Towers of Hanoi – Step by Step



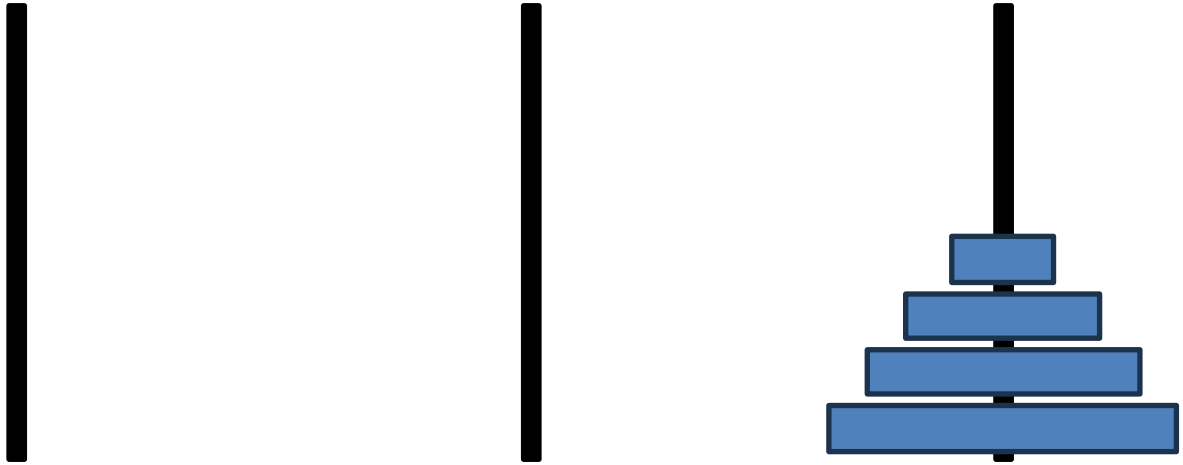
Towers of Hanoi – Step by Step



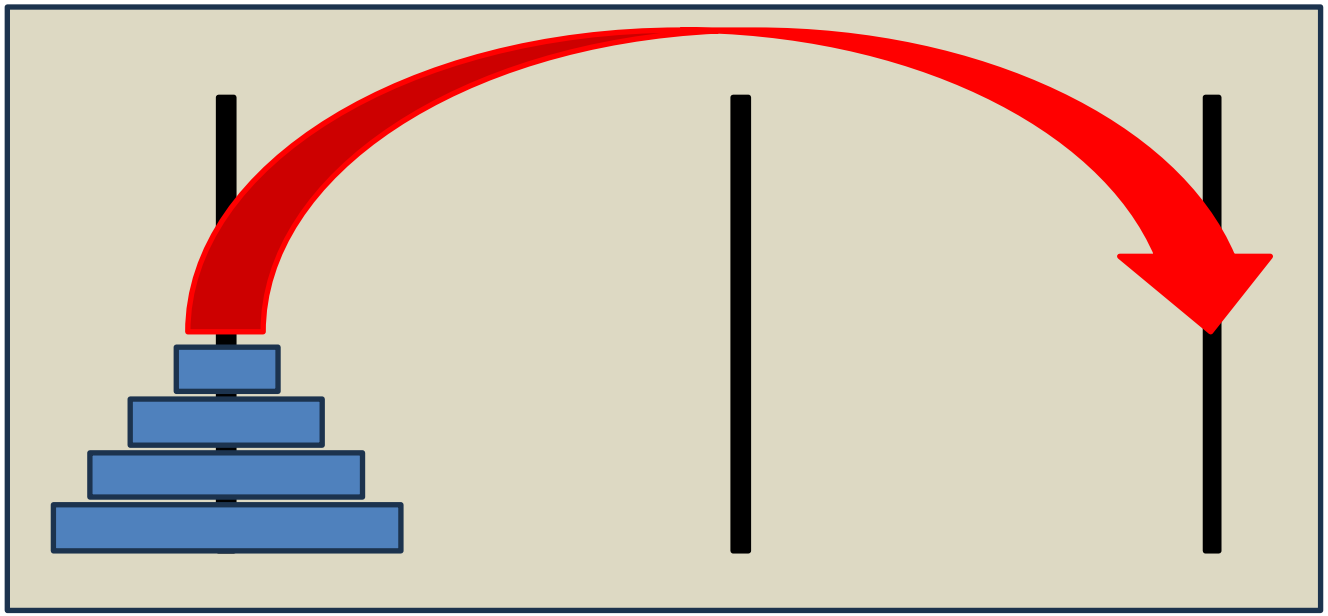
Towers of Hanoi – Step by Step



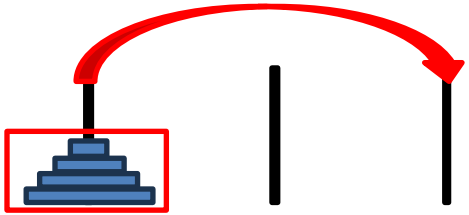
Towers of Hanoi – Step by Step



Towers of Hanoi – Step by Step



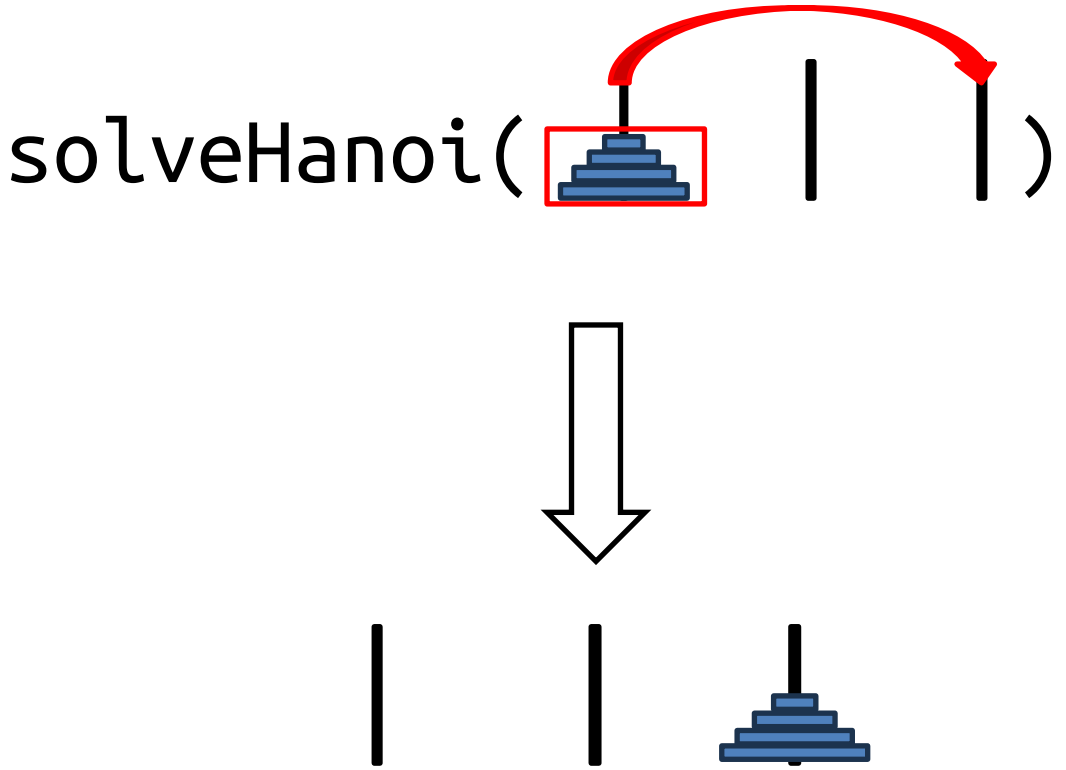
Towers of Hanoi – Methoden

Was soll `solveHanoi(`  `)` können?

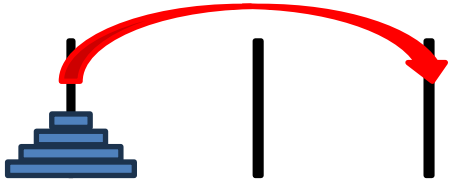
Input: Hanoi Konstellation mit zwei “freien” Stäben und ein (Teil-)stapel, der auf einen “freien” Stab bewegt werden soll.

- Ein Stab ist frei, wenn der zu bewegende Stapel nur kleinere Scheiben enthält, als die Scheiben auf dem Stab.

Towers of Hanoi – Methoden



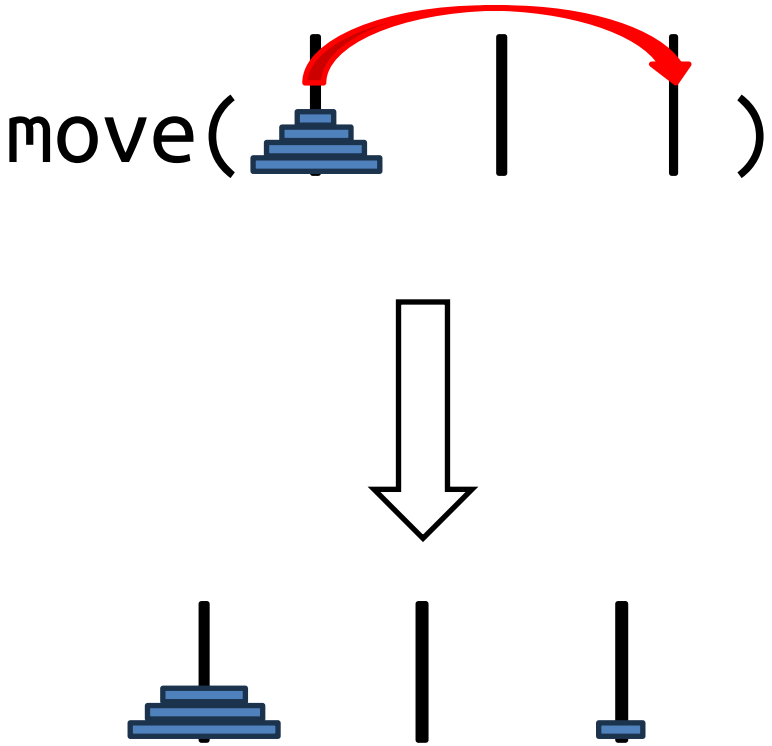
Towers of Hanoi – Methoden

Was soll `move(`  `)` können?

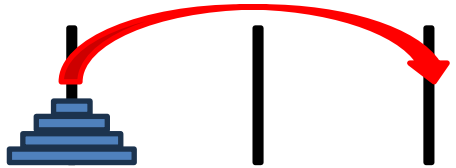
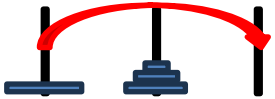
Input: Hanoi Konstellation mit mindestens einem “freien” Stab und eine Scheibe soll auf einen “freien” Stab bewegt werden.

- Ein Stab ist frei, wenn der zu bewegende Stapel nur kleinere Scheiben enthält, als die Scheiben auf dem Stab.

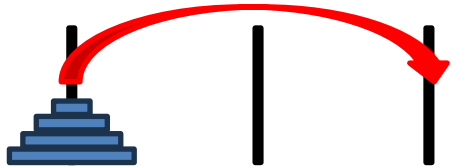
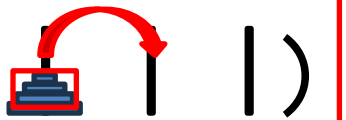
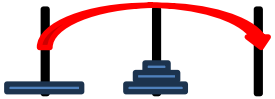
Towers of Hanoi – Methoden



Towers of Hanoi – Programm

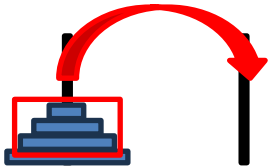
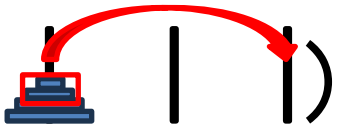
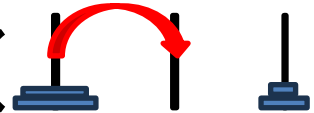
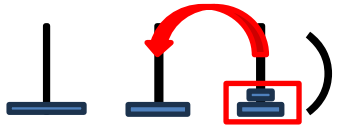
```
solveHanoi(  ) {
    solveHanoi(  )
    move(  )
    solveHanoi(  )
}
```

Towers of Hanoi – Programm

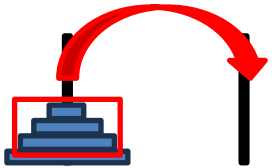
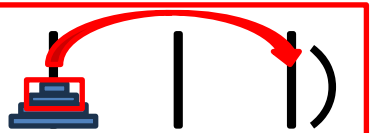
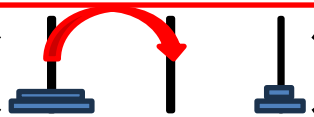
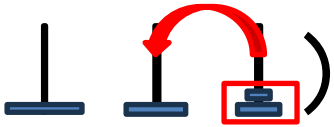
```
solveHanoi(  ) {
    solveHanoi(  )
    move(  )
    solveHanoi(  )
}
```

Wie ?

Towers of Hanoi – Programm

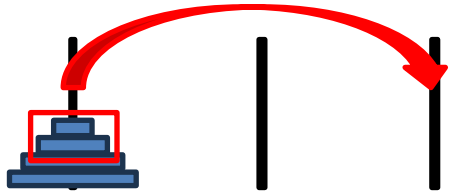
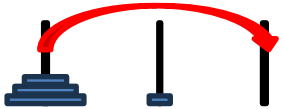
```
solveHanoi(  ) {
    solveHanoi(  )
    move(  )
    solveHanoi(  )
}
```

Towers of Hanoi – Programm

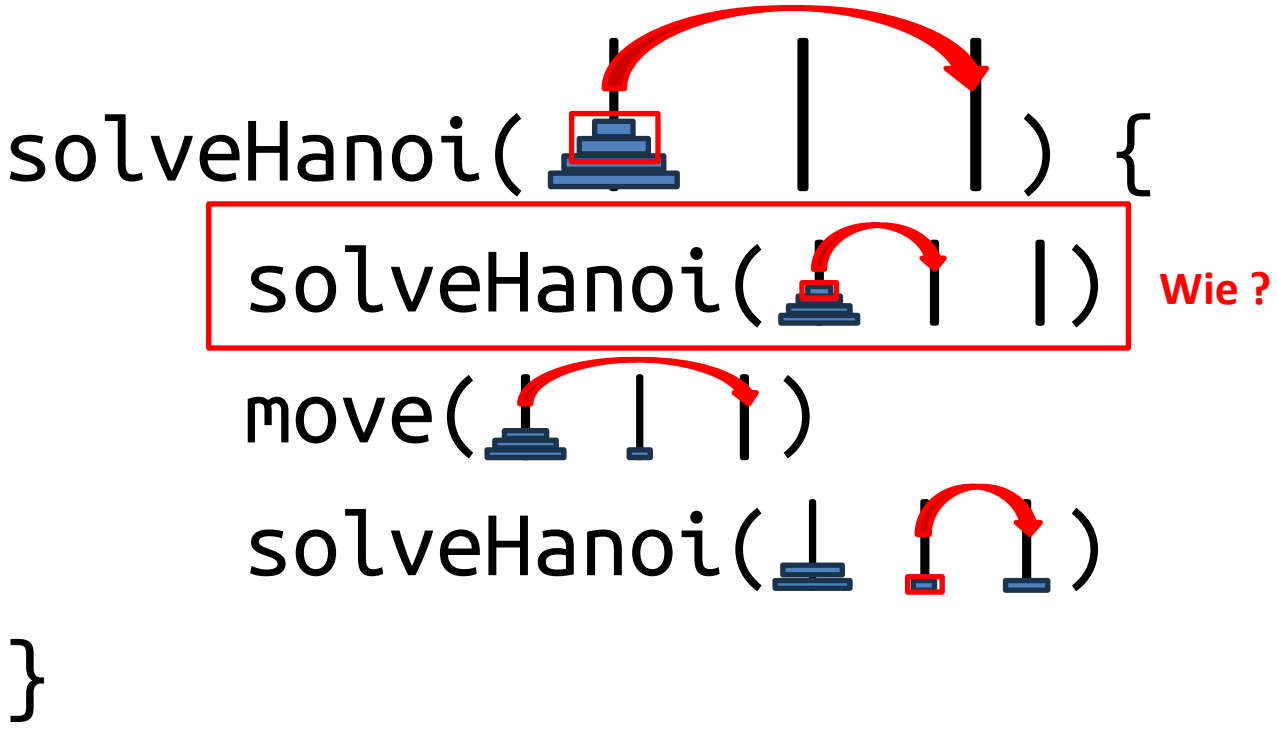
```
solveHanoi(  ) {
    solveHanoi(  )
    move(  )
    solveHanoi(  )
}
```

Wie ?

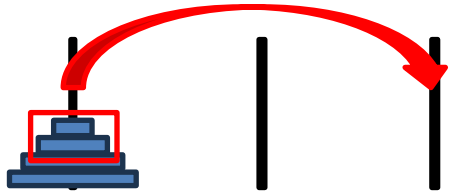
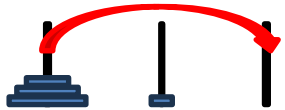
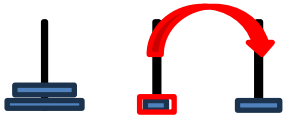
Towers of Hanoi – Programm

```
solveHanoi(  ) {
    solveHanoi(  )
    move(  )
    solveHanoi(  )
}
```

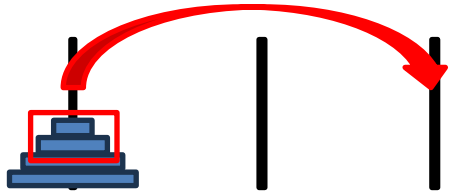
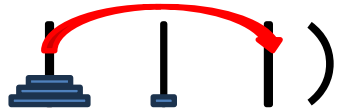
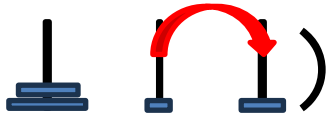
Towers of Hanoi – Programm



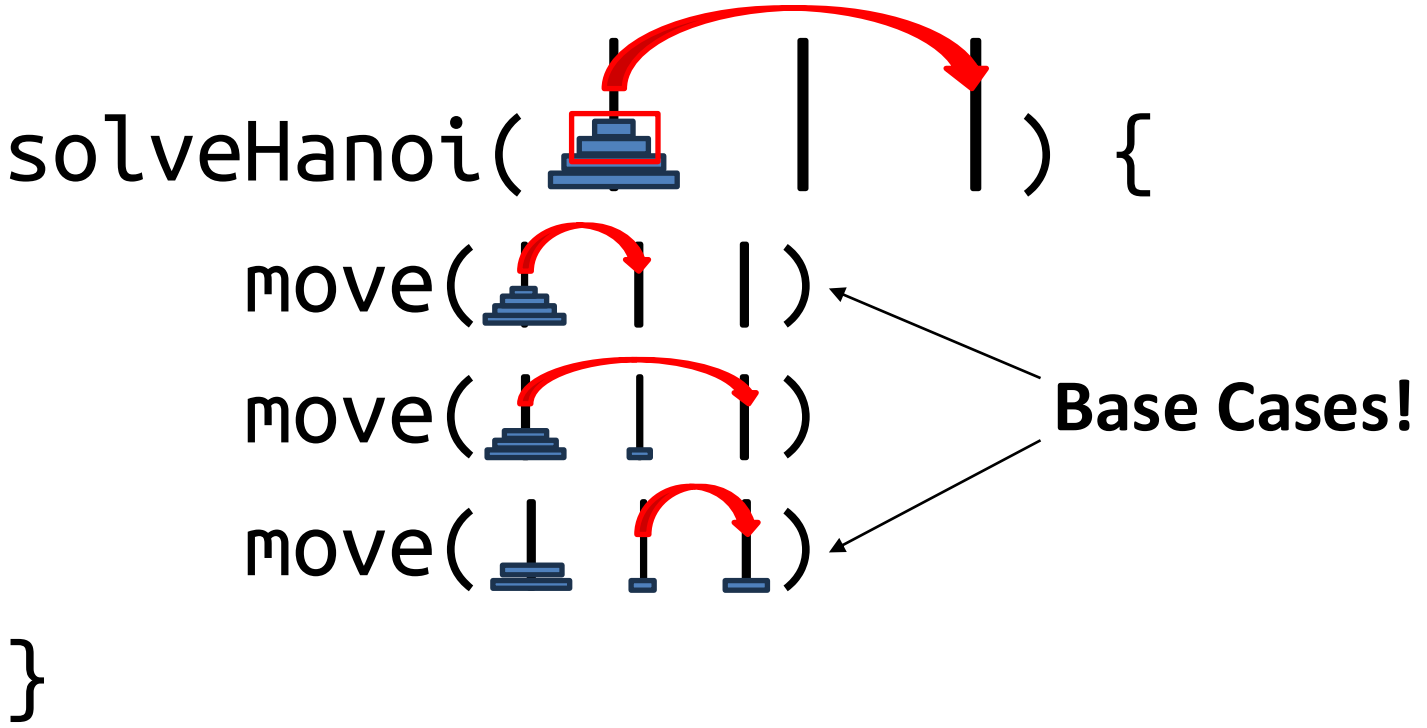
Towers of Hanoi – Programm

```
solveHanoi(  ) {
    move(  )
    move(  )
    solveHanoi(  )
}
```

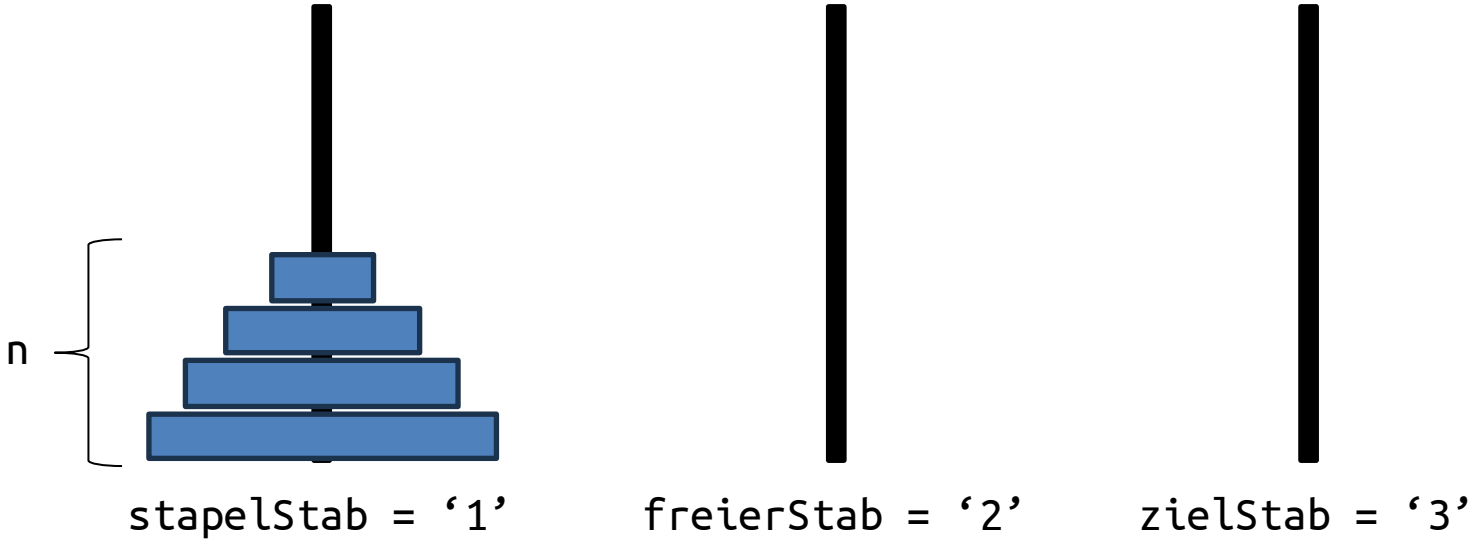
Towers of Hanoi – Programm

```
solveHanoi(  ) {
    move(  )
    move(  )
    move(  )
}
```

Towers of Hanoi – Programm



Towers of Hanoi – Als Programm



Towers of Hanoi – Als Programm

- **Methode 1:** `solveHanoi(int n, char stapelStab, char freierStab, char zielStab)`

- **Methode 2:** `move(int n, char stapelStab, char freierStab, char zielStab)`

Towers of Hanoi – Als Programm

```java
public class TowerOfHanoi {
    public static void move(int n, char stapelStab, char freierStab, char zielStab) {
        System.out.println("Scheibe " + n + " wurde von Stab " + stapelStab + " zu Stab " + zielStab + " bewegt.");
    }
    public static void solveHanoi(int n, char stapelStab, char freierStab, char zielStab) {
        if(n == 1) { // Base Case
            move(n, stapelStab, freierStab, zielStab);
        } else { // Step Case
            solveHanoi(n - 1, stapelStab, zielStab, freierStab);
            move(n, stapelStab, freierStab, zielStab);
            solveHanoi(n - 1, freierStab, stapelStab, zielStab);
        }
    }
    public static void main(String[] args) {
        int scheiben = 4;

        solveHanoi(scheiben, '1', '2', '3');
    }
}
```

Permutation Check

Beispielsaufgabe

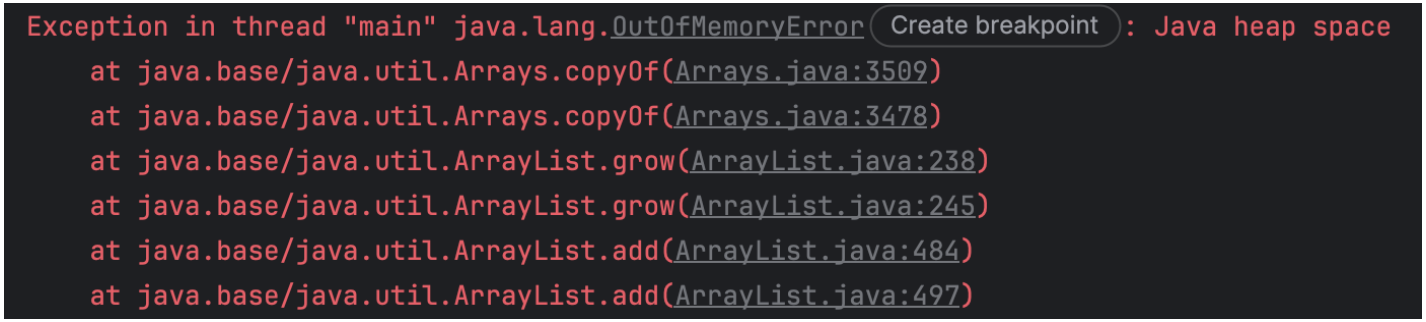
Gegeben sind die Integer Arrays `s` und `t`. Überprüfe, ob `t` eine Permutation von `s` ist.

Lösung 1: Generiere alle Permutationen von `s` und schaue ob `t` eine davon ist.

Beispielsaufgabe

Gegeben sind die Integer Arrays `s` und `t`. Überprüfe, ob `t` eine Permutation von `s` ist.

Lösung 1: Generiere alle Permutationen von `s` und schaue ob `t` eine davon ist.



Out of Memory?

- Java erhält für das Ausführen eines Programms eine **beschränkte** Menge Arbeitsspeicher (Memory).

- In der vorherigen Lösung generieren wir **O(n!) Permutationen**, welche alle Speicher benötigen.

- Irgendwann hat es **nicht mehr genug Speicher** und das Programm terminiert mit einem Laufzeit**fehler**.

Beispielsaufgabe

Gegeben sind die Integer Arrays `s` und `t`. Überprüfe, ob `t` eine Permutation von `s` ist.

Lösung 2: Generiere der Reihe nach alle Permutationen von `s` und überprüfe direkt nach dem Erstellen einer Permutation, ob sie gleich `t` ist.



Beispielsaufgabe

Gegeben sind die Integer Arrays `s` und `t`. Überprüfe, ob `t` eine Permutation von `s` ist.

Lösung 3: Prüfe ob `s` und `t` gleich lang sind und ob sie die gleichen Elemente gleich oft enthalten.

Beispielsaufgabe

Ge prüfe,
ob

Lösu b sie
die g

```java
public static boolean isPermutationSort(int[] s, int[] t) {
    if (s.length != t.length) {
        return false;
    }

    int[] sCopy = s.clone();
    int[] tCopy = t.clone();

    Arrays.sort(sCopy);
    Arrays.sort(tCopy);

    return Arrays.equals(sCopy, tCopy);
}
```

Vorbesprechung

Aufgabe 1: Dreiecksmatrix



Die Klasse `Triangle` erlaubt die Darstellung von $Z \times S$ Dreiecksmatrizen (von `int` Werten). Z und S sind immer strikt grösser als 1 (d.h., > 1). Eine $Z \times S$ Dreiecksmatrize hat Z Zeilen $X_0, X_1, ..., X_{Z-1}$, wobei Zeile X_i genau $(i * (S-1)/(Z-1)) + 1$ viele Elemente hat. Dieser Ausdruck wird nach den Regeln für `int` Au
das $(j+1)$-te Element in der $(i+$
immer genau 1 Element hat]. Abb
dass es möglich ist, dass zwei (auf

0,0

1,0 1,1

2,0 2,1 2,2 2,3

3,0 3,1 3,2 3,3 3,4

4,0 4,1 4,2 4,3 4,4

Abbildung 1: Beispiele

In der Datei `Triangle.java` finden Sie die Klasse `Triangle` mit einem Konstruktor `Triangle(int z, int s)`, der eine `z` × `s` Dreiecksmatrix erstellt. Dieser Konstruktor setzt die Werte aller Elemente auf 0. Vervollständigen Sie diese Klasse, so dass die folgenden Methoden unterstützt werden:

1. `int get(int i, int j)` gibt das Element $D_{i,j}$ zurück.
2. `void put(int i, int j, int value)` setzt das Element $D_{i,j}$ auf den Wert `value`.
3. `int[] linear()` liefert die Elemente in der kanonischen Reihenfolge (die Elemente jeder Zeile mit steigendem Index, und die Zeilen in steigender Reihenfolge).
4. `void init(int[] data)` ersetzt die Elemente von `D` durch die Werte in `data`. Sie dürfen annehmen, dass `data` genauso viele Elemente hat wie `D`. Die Methode setzt die Elemente von `D`, so dass die Folge `D.init(data); int[] y = D.linear();` in einen Array `y` resultiert für den `Arrays.equals(y, data)` den Wert `true` ergibt.
5. `void add(Triangle t)` Ein Aufruf `D.add(t)` addiert zu jedem Element $D_{i,j}$ den Wert von $t_{i,j}$, falls $t_{i,j}$ existiert. Falls $t_{i,j}$ nicht existiert, dann bleibt $D_{i,j}$ unverändert.

Tests finden Sie in der Datei "TriangleTest.java". Die Datei "TriangleGradingTest.java" enthält die Tests, welche wir bei der Prüfung für die Korrektur verwendet haben. Wir empfehlen, diese Tests erst zu verwenden, wenn Sie denken, dass Ihre Lösung korrekt ist, damit Sie sehen können, wie Sie bei einer Prüfung abgeschnitten hätten.

alte Prüfungsaufgabe



Aufgabe 2: Hoare Tripel

Hinweis: Diese Aufgabe wird als Moodle-Quiz bearbeitet.

Aufgabe 3: Weakest Precondition

Hinweis: Diese Aufgabe wird als Moodle-Quiz bearbeitet.

Aufgabe 4: Blackbox Testing

Im letzten Übungsblatt haben Sie Testautomatisierung mit JUnit kennengelernt. In dieser Aufgabe sollen Sie nun Tests für eine Methode schreiben, deren Implementierung Sie nicht kennen. Dadurch werden Sie weniger durch möglicherweise falsche Annahmen beeinflusst, die bei einer Implementierung getroffen wurden. Sie müssen sich also überlegen, wie sich *jede* fehlerfreie Implementierung verhalten muss. Diesen Ansatz nennt man auch Black-Box Testing da die Details der Implementation verdeckt sind.

In Ihrem "U06"-Projekt befindet sich eine "blackbox.jar"-Datei, welche eine kompilierte Klasse `BlackBox` enthält. Den Code dieser Klasse können Sie nicht sehen, aber sie enthält eine Methode `void rotateArray(int[] values, int steps)`, welche Sie aus einer eigenen Klasse oder einem Unit-Test aufrufen können. Diese Methode "rotiert" ein `int`-Array um eine gegebene Anzahl Schritte.

Vereinfacht macht die Methode `rotateArray()` Folgendes: Eine Rotation mit `steps=1` bedeutet, dass alle Elemente des Arrays um eine Position nach rechts verschoben werden. Das letzte Element wird dabei zum ersten. Mit `steps=2` wird alles um zwei Positionen nach rechts rotiert, usw. Eine Rotation nach links kann mit einer negativen Zahl für `steps` erreicht werden. Das folgende Beispiel ist der erste, einfache Test, den Sie in der Datei "BlackBoxTest.java" finden:

```
int[] values = new int[] { 1, 2 };
int[] expected = new int[] { 2, 1 };
BlackBox.rotateArray(values, 1);
assertArrayEquals(expected, values);
```

Aufgabe 5: Levels



Implementieren Sie die Methode `sumLevel(int[][] matrix, int level)` in der Klasse `Levels`, die für eine quadratische Matrix (Parameter `matrix`) und eine angegebene „Level"-Nummer (Parameter `level`) die Summe der Elemente dieses Levels berechnet.
Ein Level ist definiert durch die Schichten der Matrix, die von innen nach aussen gezählt werden. Level 0 entspricht dem Zentrum der Matrix. Höhere Level beschreiben jeweils die nächsten, äusseren Ränder der Matrix.

In Abbildung 2 finden Sie zwei Beispiele für Matrizen und ihre Levels. In Abbildung 2a beispielsweise besteht das innerste Level, Level 0, lediglich aus einem Element. Abbildung 2b zeigt eine Matrix mit drei Levels: Der äussere Rand der Matrix ist Level 2, der mittlere Rand Level 1 und Level 0 besteht aus den innersten vier Elementen.

1 Level: 1	2 Level: 1	3 Level: 1
4 Level: 1	5 Level: 0	6 Level: 1
7 Level: 1	8 Level: 1	9 Level: 1

(a) 3 × 3-Matrix

10 Level: 2	20 Level: 2	30 Level: 2	40 Level: 2	50 Level: 2	60 Level: 2
15 Level: 2	25 Level: 1	35 Level: 1	45 Level: 1	55 Level: 1	65 Level: 2
70 Level: 2	80 Level: 1	90 Level: 0	100 Level: 0	110 Level: 1	120 Level: 2
5 Level: 2	10 Level: 1	15 Level: 0	20 Level: 0	25 Level: 1	30 Level: 2
1 Level: 2	2 Level: 1	3 Level: 1	4 Level: 1	5 Level: 1	6 Level: 2
99 Level: 2	88 Level: 2	77 Level: 2	66 Level: 2	55 Level: 2	44 Level: 2

(b) 6 × 6-Matrix

Abbildung 2: Beispiele für gültige Matrizen und ihre Levels

alte Prüfungsaufgabe

Timed Bonus



- Release: 03.11.2026 17:00 – 19:00
- Simuliert die Prüfung unter Zeitdruck

Danke!